THE COMPLEXITY OF LOGARITHMIC SPACE BOUNDED COUNTING CLASSES TUTORIAL

Thirumalai Chakravarthi Vijayaraghavan

Ramakrishna Mission Vivekananda College
72, Sivasami Salai, Mylapore
Chennai-600004
Tamil Nadu, India.
Email: tcv.tclsbc@gmail.com

2010 *Mathematics Subject Classification. Primary:* 68Q10, 68Q15

Key words and phrases. Computational Complexity, Counting classes, Logarithmic space bounded computation

Abstract. In this manuscript, we study complexity classes that are defined using $O(\log n)$-space bounded non-deterministic Turing machines. We prove salient results of Computational Complexity in this topic such as the Immerman-Szelepcsényi Theorem, the Isolating Lemma, and many consequences of these two important results. The manuscript is intended to be a tutorial notes on the topic of The Complexity of Logarithmic Space Bounded Counting Classes.

Contents

Preface

It is a fact that the study of complexity classes has grown enormously and an innumerable number of results have been proved on almost every complexity class that has been defined. A counting class is defined as any complexity class whose definition is based on a function of the number of accepting and/or the number of rejecting computation paths of a non-deterministic space bounded Turing machine. In Chapters 1 & 2, we introduce the beautiful and sophisticated theory of Logarithmic Space Bounded Counting Complexity Classes. The definition of any Logarithmic Space Bounded Counting Complexity Class is based on the fundamental complexity class Non-deterministic Logarithmic space (NL). This complexity class NL is the first of any Logarithmic Space Bounded Counting Complexity Class. The purpose of this book is to serve as a tutorial and an introductory material for teaching the topic of logarithmic space bounded counting complexity classes highlighting some of these complexity classes and some salient results on them. In particular we study some important results on a number of complexity classes whose definitions are based on Turing machines and which are contained between the circuit complexity classes NC^1 and NC^2. The complexity classes of interest to us are NL, $\sharp$L, GapL, $C_=L$, UL, PL and ModL. Results we prove in Chapters 1 & 2 are diverse in the techniques involved such as the following:

(1) non-deterministic counting method invented in the Immerman-Szelepcsényi Theorem and some of its useful consequences which is to show that $L^{NL} = NL$ and $NL = C_=L$,

(2) using the Isolating Lemma to show that $NL/poly = (UL \cap \text{co-UL})/poly$,

(3) the double inductive counting method used to prove a combinatorial closure property of $\sharp$L under the assumption that NL = UL and its implications for ModL, and

(4) using polynomials to approximate the sign of a GapL function and its application to show closure properties of PL.

Acknowledgments

I bow before the Holy Trio of the Ramakrishna Mission for providing me with good health and strength in preparing this manuscript to fruition. I express my sincere gratitude and heartfelt thanks to Swami Dhyanagamyanandaji Maharaj, The Secretary, Ramakrishna Mission Vivekananda College, and Swami Mukthimayanandaji Maharaj for providing the necessary facilities to help me complete writing this tutorial notes. I offer my cordial thanks to my colleagues and staff members and well-wishers for their continuous encouragement.

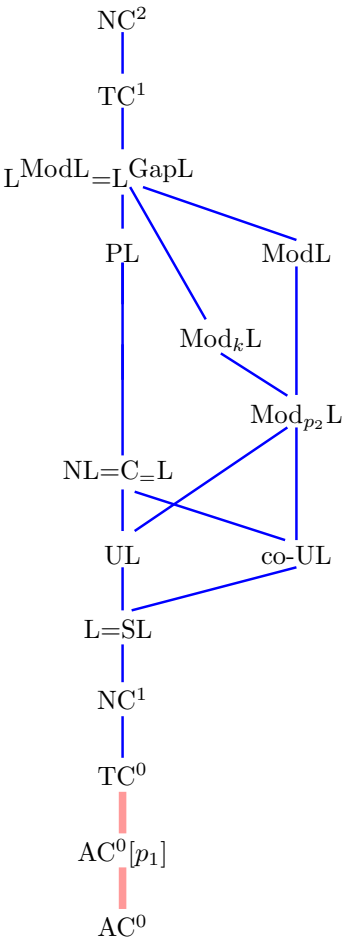
I am extremely grateful to my doctoral advisor Prof. Dr. V. Arvind for the continuous encouragement and support he has given to me in pursuing research ever since I joined the Institute of Mathematical Sciences, C.I.T. Campus, Taramani, Chennai-600113 as a research scholar in Theoretical Computer Science. His invaluable guidance, ever encouraging words and timely help have rescued me from tough situations and helped me shape this work.

I express my sincere thanks and gratitude towards the higher authorities of the Bharath Institute of Higher Education and Research (BIHER), Selaiyur, Chennai and people with whom I was interacting in BIHER, especially Mr. S. Madhan and Ms. V. Janaki for useful discussions in obtaining the result that NL = $C_{=}L$ shown in Section 1.8 of Chapter 1, and for stimulating my thoughts towards preparing the initial version of this manuscript.

I thank the American Mathematical Society (AMS) for providing LaTeXstyle files in the resources for book authors webpage provided in the their website. I thank the Association for Computing Machinery (ACM) for extending its valuable support to computing professionals.

I am extremely indebted to my mother for always being prompt and never late in cooking and providing me food everyday and in taking care and showing attention pertaining to any issue of mine if I lacked confidence and direction in tackling it.

T. C. Vijayaraghavan
January 4, 2025.



The landscape of complexity classes contained in NC^2. *In this figure, thick lines colored with pink denote strict containment and ordinary lines colored with blue denote ordinary containment and it is not known if it is strict. Here $p_1, p_2 \geq 2$ are primes and $k \geq 6$ is a composite number that has more than one distinct prime divisor and $p_2 | k$. We assume that the circuit complexity classes are uniform circuit complexity classes.*

CHAPTER 1

Introduction

1.1. The Turing machine model of computation

Here all the Turing machines that we use to prove our results have a one-way read-only input tape and a one-way read-write work tape. The input tape and the work tape are assumed to be one-way to mean that there are only finitely many tape cells to the left and that both these tapes have infinitely many cells on the right side. Both these tapes have separate tape heads. The input tape has a tape head which can move in both left and right directions. But it is not allowed to move to the left of its leftmost or the first input tape cell and this tape head is used only for reading symbols from the input tape. We follow the standard assumption that the blank symbol $\sqcup \notin \Sigma$, where Σ is the input alphabet. Assume that the input tape contains the input x from Σ^*. The input tape head is assumed not to move to the right of the right most or the last symbol of x. For the work tape also we have a tape head which can also move in both the left and right directions. But it is not allowed to move to the left of its leftmost or the first work tape cell. Using this tape head we can read and write symbols in Γ to the work tape of M, where Γ is the output alphabet. We also assume that the accepting state and the rejecting state are unique amongst the set of all states of M. Any Turing machine decides to accept or reject an input string depending on whether it enters its accepting or rejecting state. We also assume that when a Turing machine accepts its input or rejects its input by changing its state to the unique accepting state or to the unique rejecting state respectively, the contents of the work tape have been erased and that the work tape contains only the blank symbol in all its tape cells. Also M stops its computation if enters either the unique accepting or the unique rejecting state.

1.1.1. Configuration. Given a Turing machine M we define the configuration of M as follows. Let Σ be the input alphabet and Γ be the output alphabet. We assume without loss of generality that the blank symbol $\sqcup \in \Gamma$. Given an input string $x \in \Sigma^*$, we define the configuration of M on x as the tuple $(i, w_1 q w_2)$ where $q \in Q$ denotes the state of M, the number $1 \leq i \leq (n+1)$ denotes the position of the input tape head on the input tape and $w_1, w_2 \in \Gamma^*$ such that the work tape contains the string $w_1 w_2$ with the work tape head pointing to the first symbol of the string w_2. Note that we do not include the input string x as a part of the configuration of M. As a result the amount of space required to store a configuration for a $O(S(n))$-space bounded Turing machine M is $O(S(n))$ and the number of configurations that can exist is upper bounded by $n|Q||\Gamma|^{k.S(n)}$, where $k > 0$ is a constant. Specifically, when M is a NL Turing machine each configuration of M is of size $O(\log n)$ and the number of configurations of M on an input $x \in \Sigma^*$ is $\leq n^k$, where $n = |x|$ and $k > 0$ is a constant. For any non-deterministic Turing machine M that uses atmost $O(S(n))$ space, since there exists an upper bound on the number of configurations of M on any given input $x \in \Sigma^*$, we assume without loss of generality that M halts on all inputs either in the unique accepting state or in the unique rejecting state. It follows from the definition of a non-deterministic Turing machine that if M is a non-deterministic Turing machine, then from any configuration of M it is possible to move to more

than one configuration based on the state of M, the symbol read by the input tape head and the symbol read by the work tape head.

1.1.2. Computation tree. It is easy to observe that the computation of a $O(S(n))$-space bounded non-deterministic Turing machine M on a given input is in fact a tree which we call the computation tree of M. Since there is an upper bound on the number of configurations of a $O(S(n))$-space bounded non-deterministic Turing machine the computation tree is finite containing finite number of nodes and edges in it.

Since the size of the input alphabet, the output alphabet and the number of states of a $O(S(n))$-space bounded non-deterministic Turing machine M is a constant it is possible to assume that when M makes a non-determinisitc choice in its computation, the number of branches in the computation tree is exactly 2. As a result we assume without loss of generality that computation tree of M is a binary tree. Clearly if M is a NL-Turing machine then the depth of its binary computation tree of M is a polynomial in the size of the input and the number of nodes it has is exponential in the input size. From now onwards in the rest of this manuscript, we assume that the computation tree is always a binary tree.

In any such computation tree of a $O(S(n))$-space bounded non-deterministic Turing machine, we assume without loss of generality that all the computation paths of M on an input of length n have the same length. In other words, computation proceeds for the same number of steps on any computation path in the computation tree of a non-deterministic Turing machine.

Now without loss of generality assume that the input alphabet of M, denoted by Σ, and the output alphabet of M, denoted by Γ, are the same. Assume that we have a function $g : \Sigma^* \to \Gamma^*$. When we say that g is computable by a $O(S(n))$-space bounded non-deterministic Turing machine M we mean that given an input $x \in \Sigma^*$, we check if the i-th symbol of $g(x)$ is $a \in \Gamma^*$ by submitting the triple (x, i, a) as an input to M and finding out if M enters its accepting or rejecting configuration in its computation, where $1 \leq i \leq |g(x)|$. Therefore by submitting various input instances (x, i, a) to M for different values of $a \in \Gamma$, it is possible to find out the correct value of the i-th symbol of $g(x)$, given $x \in \Sigma^*$.

1.1.3. Oracle Turing machines. If M^A is a Turing machine that has access to an oracle A then M^A has a separate one-way write-only oracle tape in which it can submit queries to the oracle A. As in the case of the input tape and the work tape of M^A, the oracle tape of M^A is assumed to be one-way to mean that there are only finitely many tape cells to the left and that this tape also has infinitely many cells on the right side. The oracle tape of M^A has a tape head which can move in only one direction from the left to the right. Also M^A has three exclusive states q_{QUERY}, q_{YES} and q_{NO}. When M^A completes writing an oracle query string $y \in \Gamma^*$ in the oracle tape of A, it enters the q_{QUERY} state. In the next instant the oracle A erases the contents of the oracle tape, brings the oracle tape head to the left most cell of the oracle tape and changes the state of M^A from q_{QUERY} to either q_{YES} or q_{NO} depending on whether $y \in A$ or $y \notin A$ respectively.

1.1.4. Non-deterministic oracle Turing machines and the Ruzzo-Simon-Tompa oracle access mechanism.

1.1.4.1. *Model 1.* If M^A is a non-deterministic Turing machine then we follow the *Ruzzo-Simon-Tompa oracle access mechanism* in which we assume that M^A always submits its queries to A in a deterministic manner only. This means that all the queries of M^A are submitted to A after doing some computation and even before the first non-deterministic choice has been made in the computation on the given input. We note that the oracle tape is a write only tape of M^A. As a result contents of the oracle tape are never read by M^A

to generate a query submitted to A. Therefore if M^A is a $O(S(n))$ space bounded non-deterministic Turing machine then the number of configurations of M^A is also upperbounded by $n|Q||\Gamma|^{k.S(n)}$, where $k > 0$ is a constant. From this we infer that the length of any query and the number of queries that M^A can submit to the oracle A are both at most $n|Q||\Gamma|^{k.S(n)}$, where $k > 0$ is a constant.

1.1.4.2. *Model 2: Oblivious oracle Turing machines.* Due to these upper bounds and the Ruzzo-Simon-Tompa oracle access mechanism we can also suggest a modification to this model of non-deterministic oracle Turing machine described above that is also $O(S(n))$ space bounded and in which we consider all the configurations of M^A that will start to submit a query to the oracle A starting from the lexicographically least such configuration. Starting from the lexicographically least such configuration we generate all the $n|Q||\Gamma|^{k.S(n)}$ queries, where $k > 0$ is a constant, in succession and write them onto the one-way write only oracle tape separating any two adjacent queries using a special symbol, namely # that is not there previously in Γ and which we now include in Γ. We in fact iterate this process of submitting the $n|Q||\Gamma|^{k.S(n)}$ many queries, where $k > 0$ is a constant, $n|Q||\Gamma|^{k.S(n)}$ many times. After submitting these queries, M^A moves to the q_{QUERY} state and in the next instant the oracle A erases the contents of the oracle tape, brings the oracle tape head to the left most cell of the oracle tape and replaces each query with the oracle reply of either 0 if the oracle query string is not in A or with 1 if the oracle query string is in A. In addition, replies given by the oracle for any two adjacent oracle queries are separated by $\# \in \Gamma$. Clearly this procedure of M^A submitting queries to the oracle A and obtaining the reply of the oracle A does not involve any non-determinism and so we have followed the Ruzzo-Simon-Tompa oracle access mechanism in submitting queries to A and obtaining replies from A. Now the non-deterministic oracle Turing machine M^A performs its computation on the input string x, where $n = |x|$. When it generates the i-th query, M^A finds it to be the j-th query amongst all the queries that it can generate and it looks at the j-th reply of A in the i-th set of queries submitted by M^A in the oracle tape. If the oracle reply is 0 then the oracle changes the state of M^A to q_{NO} since the oracle query string is not in A. Otherwise if the reply is 1 then the oracle changes the state of M^A to q_{YES} since the oracle query string is in A. Having obtained this reply of the oracle A, the Turing machine M^A continues with its computation and finally ends its computation path either in the accepting configuration or in the rejecting configuration. We call such oracle Turing machines as Oblivious oracle Turing machines since the queries submitted to the oracle depend only on the length of the input string in the input tape and not on the input string itself.

NOTE 1. **In this manuscript, any result involving non-deterministic oracle Turing machines always uses the model of computation which is Model 1 described in Section 1.1.4.1 unless otherwise specified.**

1.2. NL: Nondeterministic Logarithmic Space

DEFINITION 1.1. Let Σ be the input alphabet. For a language $L \subseteq \Sigma^*$, the complement of L is $\overline{L} = \Sigma^* - L$. Given a complexity class $\mathcal{C}$, we define the complement of $\mathcal{C}$ as co-$\mathcal{C}$ $= \{\overline{L} \subseteq \Sigma^* | L \in \mathcal{C}\}$.

DEFINITION 1.2. Let Σ be the input alphabet. We define the complexity class NL $= \{L \subseteq \Sigma^* |$ there exists a nondeterministic $O(\log n)$ space bounded Turing machine M such that $L = L(M)\}$.

DEFINITION 1.3. Let Σ be the input alphabet. For a language $L \in \Sigma^*$, the complement of L is $\overline{L} = \Sigma^* - L$. We define the complexity class co-NL $= \{\overline{L} \subseteq \Sigma^* |$ there exists a nondeterministic $O(\log n)$ space bounded Turing machine M such that $L = L(M)\}$.

DEFINITION 1.4. Let Σ be the input alphabet and $L_1, L_2 \in \Sigma^*$. We say that L_1 is logspace many-one reducible to L_2, denoted by $L_1 \leq_m^{\mathrm{L}} L_2$, if there exists a function $f : \Sigma^* \to \Sigma^*$ such that given any input string $x \in \Sigma^*$, we have $f(x) \in L_2$ if and only if $x \in L_1$.

It is easy to note that if L_1 is logspace many-one reducible to L_2 and L_2 is in a complexity class $\mathcal{C}$ where $\mathcal{C}$ contains the complexity class L of all languages that can be decided in deterministic logarithmic space then L_1 is in $\mathcal{C}$.

DEFINITION 1.5. Let Σ be the input alphabet and let $\mathcal{C}$ be a complexity class. Then $L \subseteq \Sigma^*$ is logspace many-one hard for $\mathcal{C}$ if for any $L' \in \mathcal{C}$, we have $L' \leq_m^{\mathrm{L}} L$.

DEFINITION 1.6. Let Σ be the input alphabet and let $\mathcal{C}$ be a complexity class. Then $L \subseteq \Sigma^*$ is logspace many-one complete for $\mathcal{C}$ if $L \in \mathcal{C}$ and L is logspace many-one hard for $\mathcal{C}$.

1.2.1. NL and the Directed st-Connectivity Problem (DSTCON).

DEFINITION 1.7. Let $G = (V, E)$ be a directed graph and let $s, t \in V$. We define DSTCON $= \{(G, s, t) | \exists$ a directed path from s to t in $G\}$.

LEMMA 1.8. DSTCON *is hard for* NL *under* $\leq_m^{\mathrm{L}}$ *reductions.*

PROOF. Let Σ be the input alphabet and let $L \subseteq \Sigma^*$. Assume that $L \in$ NL and let M be a non-deterministic Turing machine that accepts L using space atmost $O(\log n)$ where n is the size of the input. We show that $L \leq_m^{\mathrm{L}}$ DSTCON. Given an input string $x \in \Sigma^*$, we consider the set of all configurations of M on an input of length $n = |x|$. It is easy to note that the number of configurations of M on a string of length n is atmost n^k for some constant $k > 0$. We now define a directed graph $G = (V, E)$ where the set of all vertices V is the set of all configurations of M and there exists an edge from a vertex u to another vertex v in G if the tape head upon reading the symbol it points to in configuration u results in the configuration v of M. Since $|V| \leq n^k$ and M is $O(\log n)$ space bounded, any configuration of M is also $O(\log n)$ space bounded and so given a configuration u of M on x it is always possible to find the neighbours of u in G in $O(\log n)$ space. Also given $x \in \Sigma^*$ we have $x \in L$ if and only if there exists a directed path from the vertex that is the starting configuration of M on input x to the vertex which is the accepting configuration of M on input x. This shows that $L \leq_m^{\mathrm{L}}$ DSTCON which implies that DSTCON is hard for NL under $\leq_m^{\mathrm{L}}$ reductions. □

LEMMA 1.9. DSTCON $\in$ NL.

PROOF. Let $G = (V, E)$ be a directed graph that is represented as adjacency matrix and let $s, t \in V$ be given as input along with G. The non-deterministic Turing machine that implements the above Directed-Graph-Accessibility(G, s, t) algorithm needs atmost $O(\log n)$ space to store vertices $u, v \in V$, to store variables n, m and to update the variable *counter*. In addition it looks up to the adjacency matrix of the directed graph given as input to find and non-deterministically choose vertex v which is the head of the directed edge (u, v) in each iteration. Clearly this also needs atmost $O(\log n)$ space.

Now, if there exists a directed path from s to t then any such path is of length less than or equal to $(n-1)$ and it is possible to non-deterministically guess the vertices along a directed path from s to t. Thus whenever there exists a directed path there exists a computation path of the computation tree of this non-deterministic Turing machine that accepts. Conversely, if there is no path from s to t in G then no computation path of this non-deterministic Turing machine ends in an accepting state. Therefore DSTCON $\in$ NL. □

Algorithm 1 Directed-Graph-Accessibility(G, s, t)

Let $n \leftarrow |V|$.
Let $m \leftarrow |E|$.
Let $u \leftarrow s$.
for *counter* $\leftarrow 0$ to $(n-1)$ **do**
    Non-deterministically choose a vertex $v \in V$ such that $(u, v) \in E$.
    if $\nexists v \in V$ such that $(u, v) \in E$ **then** reject the input.
    if $v = t$ **then**
        output there exists a directed path from s to t in G.
        accept the input.
    $u \leftarrow v$.
    counter $\leftarrow$ *counter* $+ 1$.
reject the input and stop.

THEOREM 1.10. DSTCON *is complete for* NL *under* $\leq_m^{\mathrm{L}}$ *reductions.*

PROOF. The result follows from Lemma 1.9 and Lemma 1.8. □

DEFINITION 1.11. We define SLDAG $= \{G | G = (V, E)$ is a simple layered directed acyclic graph which does not have any self-loops on vertices or directed cycles or parallel edges between vertices. Also vertices in G are arranged as a square matrix such that there are n rows and every row has n vertices. Any edge in this graph is from a vertex in the i^{th} row to a vertex in the $(i+1)^{st}$ row, where $1 \leq i \leq (n-1)$ and $n \geq 2\}$.

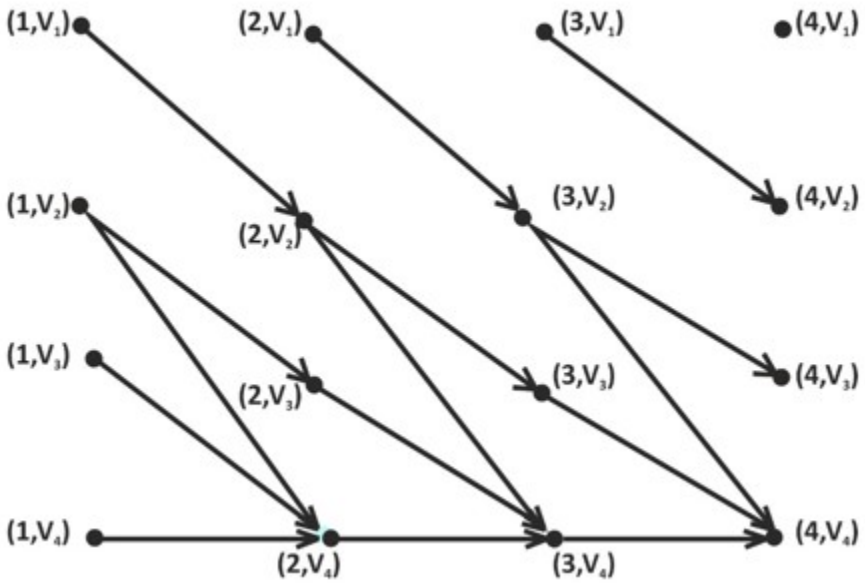


FIGURE 1. *The above figure is a directed graph* G *which is an instance of* SLDAG. *Assuming that vertices* $s = (1, V_1)$ *and* $t = (4, V_4)$ *we also infer that this instance* (G, s, t) *is in* SLDAGSTCON.

DEFINITION 1.12. Let Σ be the input alphabet. The complexity class $\sharp$L is defined to be the class of functions $f : \Sigma^* \to \mathbb{Z}^+$ such that there exists a NL Turing machine M for which we have $f(x) = acc_M(x)$ where $acc_M(x)$ denotes the number of accepting computation paths of M on any input $x \in \Sigma^*$.

DEFINITION 1.13. Let $G = (V, E) \in$ SLDAG and $s, t \in V$ such that s is a vertex in the first row and t is a vertex in the last row of G. We define SLDAGSTCON $= \{(G, s, t) | \exists$ a directed path from s to t in $G\}$.

THEOREM 1.14. SLDAGSTCON *is complete for* NL *under logspace many-one reductions.*

PROOF. We know from Theorem 1.10 that the *st*-connectivity problem for directed graphs is complete for NL under logspace many-one reductions. Therefore the *st*-connectivity problem for directed graphs that

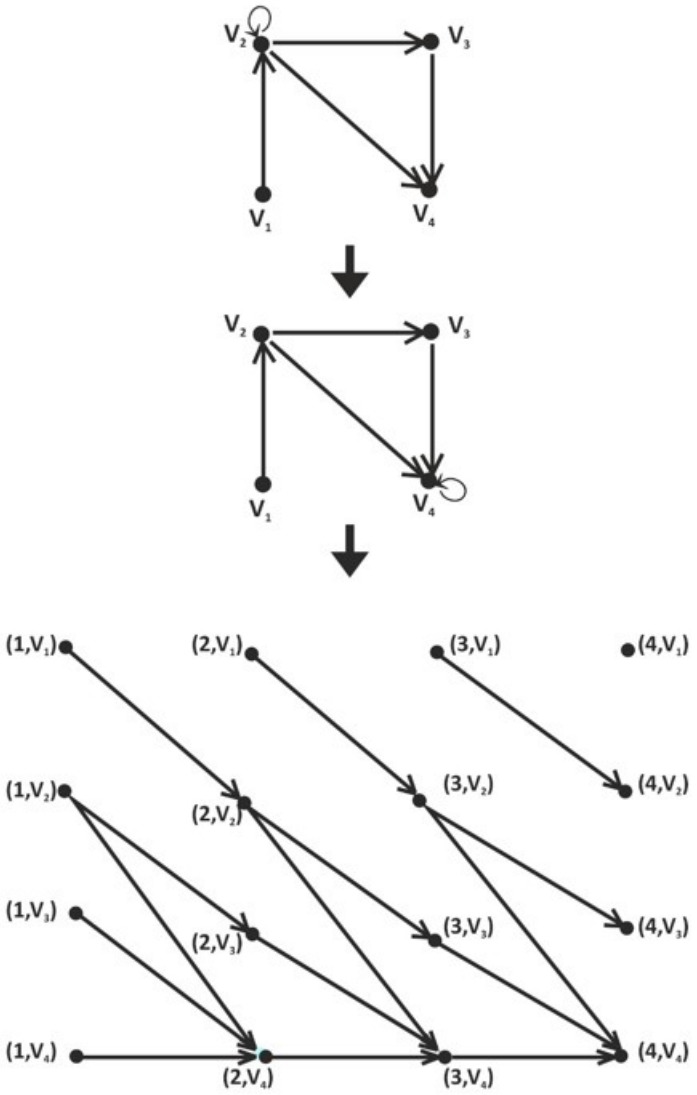


FIGURE 2. *The above figure explains the reduction from a directed graph G_1 with vertices $s_1 = V_1$ to the vertex $t_1 = V_4$ to a "yes" instance G of* SLDAGSTCON *where $s = (1, V_1)$ and $t = (4, V_4)$.*

do not have any self-loops on vertices or parallel edges between vertices is also complete for NL under logspace many-one reductions. In other words the *st*-connectivity problem for simple directed graphs is complete for NL under logspace many-one reductions. Let $G = (V, E)$ be a simple directed graph and $s, t \in V$. Also let $n = |V|$. Given G, we obtain the directed graph $G' = (V', E')$, where $V' = V$ and $E' = E \cup (t, t)$. We now reduce G' to $G'' = (V'', E'') \in$ SLDAG. In G'', the vertex set V'' has n^2 vertices arranged as a $n \times n$ matrix obtained by creating n copies of V'. As an example refer to the figure given below in which we obtain an instance of SLDAG from the directed graph given as input. For the sake of convenience, we assume in this example that the vertices in each row of the instance of SLDAG are arranged vertically.

Now let the vertex s in the first row be denoted by $(1, s)$ and the vertex t in the last row be denoted by (n, t). A (i, j) vertex in G'' is the j^{th} vertex in the i^{th} row of G'', where $1 \leq i, j \leq n$. Here $E'' = \{((i, j_1), (i + 1, j_2))|(j_1, j_2) \in E',$ where $1 \leq i \leq n - 1$ and $1 \leq j_1, j_2 \leq n\}$. Now it is easy to note that there exists a directed path from s to t in G if and only if there exists a directed path from $(1, s)$ to (n, t) in G''. Since we can obtain G'' from G using a logspace many-one reduction the result follows. □

COROLLARY 1.15. *Let (G, s, t) be an input instance of* SLDAGSTCON. *Then counting the number of directed paths from s to t in G is complete for* ♯L *under logspace many-one reductions.*

COROLLARY 1.16. *Let Σ be the input alphabet and $f \in$ ♯L using a* NL *machine M such that $f(x) = acc_M(x)$ on any input $x \in \Sigma^*$. Then for an input string $x \in \Sigma^*$, $g(x) = (G, s, t)$, where g is the canonical logspace many-one reduction used to show that* SLDAGSTCON *is* NL*-complete and (G, s, t) is an input instance of* SLDAGSTCON*, and $f(x)$ is equal to the number of directed paths from s to t in G.*

1.3. Reducibilities based on oracles

DEFINITION 1.17. Let Σ be the input alphabet and $L_1, L_2 \in \Sigma^*$. We say that L_1 is logspace Turing reducible to L_2, denoted by $L_1 \leq_T^L L_2$, if there exists a $O(\log n)$ space bounded Turing machine M that has L_2 as an oracle such that given any input string $x \in \Sigma^*$, we have $M^{L_2}(x)$ accepts if and only if $x \in L_1$.

DEFINITION 1.18. Let Σ be the input alphabet and let $\mathcal{C}$ be a complexity class. We define $L^{\mathcal{C}}$ as the class of all languages $L \subseteq \Sigma^*$ that is accepted by a $O(\log n)$ space bounded deterministic Turing machine that has oracle access to a language $A \in \mathcal{C}$, where n is the size of the input and $A \subseteq \Sigma^*$.

It is easy to note that if L_1 is logspace Turing reducible to L_2 and L_2 is in a complexity class $\mathcal{C}$ where $\mathcal{C}$ contains the complexity class L of all languages that can be decided in deterministic logarithmic space then L_1 is in $L^{\mathcal{C}}$.

DEFINITION 1.19. Let Σ be the input alphabet and let $\mathcal{C}$ be a complexity class. Then $L \subseteq \Sigma^*$ is logspace Turing hard for $\mathcal{C}$ if for any $L' \in \mathcal{C}$, we have $L' \leq_T^L L$.

DEFINITION 1.20. Let Σ be the input alphabet and let $\mathcal{C}$ be a complexity class. Then $L \subseteq \Sigma^*$ is logspace Turing complete for $\mathcal{C}$ if $L \in \mathcal{C}$ and L is logspace Turing hard for $\mathcal{C}$.

DEFINITION 1.21. Let Σ be the input alphabet and let $\mathcal{C}$ be a complexity class. We define $NL^{\mathcal{C}}$ to be the class of all languages $L \subseteq \Sigma^*$ that is accepted by a $O(\log n)$ space bounded non-deterministic Turing machine M that has oracle access to a language $A \in \mathcal{C}$, where n is the size of the input and $A \subseteq \Sigma^*$. Here we assume that M makes queries to the oracle A according to the Ruzzo-Simon-Tompa oracle access mechanism.

1.4. The Immerman-Szelepcsényi Theorem

1.4.1. The Immerman-Szelepcsényi Theorem in the logarithmic space setting.

THEOREM 1.22. **NL = co-NL**

PROOF. Let $G = (V, E)$ be an input instance of SLDAG that is given as an adjacency matrix and let $s, t \in V$ such that s is a vertex in the first row and t is a vertex in the last row of G. We know from Theorem 1.14 that deciding if $(G, s, t) \in$ SLDAGSTCON is complete for NL under logspace many-one reductions. Therefore to prove our result it suffices to prove that the problem of deciding if there does not exist any path from s to t in G is also in NL. In other words we show that $\overline{\text{SLDAGSTCON}} \in$ NL. Now let us consider the algorithm IS-Logspace(G, s, t).

Algorithm 2 IS-Logspace(G, s, t)

1: Let $\alpha \leftarrow 1$.
2: **for** $i \leftarrow 1$ to $(n-1)$ **do**
3: Let $\beta \leftarrow 1$.
4: Let $d \leftarrow 0$.
5: $flag \leftarrow 0$.
6: **for** each node v in layer $(i+1)$ in G **do**
7: **for** each node u in layer i in G **do**
8: Non-deterministically either perform or skip {step 9 to step 13}.
9: Non-determinstically follow a path of length $\leq (i-1)$ from s.
10: **if** this path does not end at u **then** reject.
11: $d \leftarrow d + 1$.
12: **if** (u, v) is an edge of G and $flag = 0$ **then**
13: $\beta \leftarrow \beta + 1$ and $flag \leftarrow 1$.
14: **if** $d \neq \alpha$ **then** reject.
15: $flag \leftarrow 0$, $\alpha \leftarrow \beta$.
16: Let $d \leftarrow 0$.
17: **for** each node u in layer n in G **do**
18: Non-deterministically either perform or skip {step 19 to step 22}.
19: Non-deterministically follow a path of length $(n-1)$ from s.
20: **if** this path does not end at u **then** reject.
21: **if** $u = t$ **then** reject
22: $d \leftarrow d + 1$.
23: **if** $d \neq \alpha$ **then** reject
24: **else** accept.

We show that the above algorithm can be implemented by a NL Turing machine M. Then M must have at least one accepting computation path whenever there does not exist any directed path from s to t in G.

Now assume that we know α which is the number of nodes in layer n of G that are reachable from from s in G. We assume that α is provided as an input to M and we show in steps 16 to 24 of the above algorithm IS-Logspace(G, s, t), that it is possible to use α to prove that $\overline{\text{SLDAGSTCON}} \in$ NL using a method which

we call as ***non-deterministic counting***. Later we prove that M also computes α in steps 1 to 15 using the same non-deterministic counting method.

Given G, s, t and α, the Turing machine M operates as follows. One by one M goes through all the n nodes in layer n of G and non-deterministically guesses whether each one is reachable from s. Whenever a node u is guessed to be reachable, M attempts to verify this guess by guessing a path of length n or less from s to u. If this computation branch fails to verify this guess, it rejects. The Turing machine M counts the number of nodes that have been verified to be reachable. When a branch has gone through all of G's nodes, it checks that the number of nodes that are verified to be reachable from s equals α, the number of nodes that are actually reachable from s, and rejects if not. Otherwise this computation branch accepts.

In other words, if M non-deterministically selects exactly α nodes reachable from s, not including t, and proves that each is reachable from s by guessing the path, M knows that the remaining nodes including t are not reachable and therefore it can accept.

Next we show in steps 1 to 15 described in the above algorithm IS-Logspace(G, s, t), how to calculate α: the number of nodes that are reachable from s in layer n in G. We describe a non-deterministic logspace procedure whereby at least one computation branch has at least the correct value of α in which it accepts and all the other branches reject.

Let $A_0 = \{s\}$. For each i from 1 to $(n-1)$, we define A_i to be the collection of all nodes in layer $(i+1)$ that are reachable from s by a directed path of length i. So A_{n-1} is the set of all nodes in layer n that are reachable from s by a directed path of length $(n-1)$. Let $\beta = |A_{i-1}|$. Then in steps 6 to 15 of the IS-Logspace(G, s, t) algorithm we show how to calculate $|A_i|$ from $|A_{i-1}|$. Repeated application of this procedure yields the desired value of α.

We calaulate $|A_i|$ from $|A_{i-1}|$ using an idea similar to the one presented earlier in this proof. In the algorithm we go through all the nodes of G in layer $(i+1)$ and determine whether each is a member of A_i and also count the members to find $|A_i|$.

To determine whether a node v is in A_i, we use an inner loop to go through all the nodes of G in layer i and guess whether each node is in A_{i-1}. Each positive guess is verified by a path of length i from s. For each node u verified to be in A_{i-1}, the algorithm tests whether (u, v) is an edge of G. If it is an edge, v is in A_i. Additionally the number of nodes verified to be in A_{i-1} is counted. At the completion of the inner loop, if the total number of nodes verified to be in A_{i-1} is not α then entire A_{i-1} has not been found, so this computation branch rejects. If the count equals α and v has not shown to be in A_i, we conclude that it is not in A_i. Then we go on to the next v in layer $(i+1)$ in the outer loop. Since we use constant number of variables each of which takes values from 0 to n in the above algorithm IS-Logspace(G, s, t), it is easy to note that the Turing machine M that implements it is a NL Turing machine. As a result we have shown that $\overline{\text{SLDAGSTCON}} \in$ NL which implies co-NL $\subseteq$ NL from which the result follows. □

THEOREM 1.23. $\mathbf{L^{NL} = NL}$

PROOF. It is trivial to note that NL $\subseteq \mathrm{L}^{\mathrm{NL}}$. Now to prove our result we have to show that $\mathrm{L}^{\mathrm{NL}} \subseteq$ NL. Let $L' \in \mathrm{L}^{\mathrm{NL}}$. Then there exists a NL oracle Turing machine M that has access to a language $A \in$ NL as an oracle such that given any string $x \in \Sigma^*$, M^A decides if $x \in L'$ correctly. Since M is $O(\log n)$ space bounded, we infer that the number of queries that M submits to the oracle A is a polynomial in the size of the input. Let M_A be the NL machine that decides if an input string is in A or not. Due to Theorem 1.22 it follows that $\overline{A}$ is also in NL. Let $M_{\overline{A}}$ be the NL machine that correctly decides if any input string is in $\overline{A}$.

Now let Σ be the input alphabet and let $x \in \Sigma^*$ be the input string. Let us consider the following algorithm implemented by a Turing machine N.

1. N simulates M on x until a query y is generated.

2. N simulates M_A on input y.

3. If M_A accepts y then N continues to simulate M on x assuming the reply of the oracle is "yes". Otherwise if M_A rejects y then N simulates $M_{\overline{A}}$ on y. If $M_{\overline{A}}$ accepts y then N assumes the reply of the oracle is "no" and continues to simulate M on x. However if both M_A and $M_{\overline{A}}$ reject y then N rejects the input string x.

We now show that N correctly decides if $x \in L$. It is easy to note that if for any oracle query sring y, either M_A or $M_{\overline{A}}$ can accept y but not both. Also if M_A accepts y then $y \in A$. Similarly if $M_{\overline{A}}$ accepts y then $y \in \overline{A}$. As a result when either of these two NL machines accept then this can be taken to be the output of the oracle and N will proceed with its simulation of M. On the contrary if neither M_A nor $M_{\overline{A}}$ accept since NL is closed under complement we know that there exists at least one accepting computation path in exactly one of these Turing machines and we did not simluate both the Turing machines along any such path. Therefore N also rejects x and stops the computation. Since N simulates only NL Turing machines, it shows that N is also a NL Turing machine. This shows that $\mathrm{L}^{\mathrm{NL}} \subseteq \mathrm{NL}$ from which we get that $\mathrm{L}^{\mathrm{NL}} = \mathrm{NL}$. □

Now the following result is easy to prove.

PROPOSITION 1.24. NL *is closed under union and intersection.*

THEOREM 1.25. $\mathbf{NL^{NL}} = \mathbf{NL}$

PROOF. It is trivial to note that $\mathrm{NL} \subseteq \mathrm{NL}^{\mathrm{NL}}$. Therefore we have to prove that $\mathrm{NL}^{\mathrm{NL}} \subseteq \mathrm{NL}$. Let $L \in \mathrm{NL}^{\mathrm{NL}}$. Then there exists a NL oracle Turing machine M that has access to a language $A \in \mathrm{NL}$ as an oracle such that given any string $x \in \Sigma^*$, M^A decides if $x \in L$ correctly. It follows from Section 1.1.4.1 that this NL oracle Turing machine follows the Ruzzo-Simon-Tompa oracle access mechanism to submit queries to the oracle A. Also we infer from Section 1.1.4.1 that the number of queries that are submitted to the oracle A is a polynomial in the size of the input. Let M_A be the NL machine that decides if an input string is in A or not. Since we have shown in Theorem 1.22 that NL is closed under complement, it follows that $\overline{A}$ is also in NL. Let $\overline{M_A}$ be the NL machine that correctly decides if any input string is in $\overline{A}$. Now let Σ be the input alphabet and let $x \in \Sigma^*$ be the input string. Let us consider the following algorithm implemented by a Turing machine N:

1. N simulates M on x until a query y is generated.

2. N simulates M_A on input y. If M_A accepts y then N continues to simulate M on x assuming the reply of the oracle is "yes". Otherwise if M_A rejects y then N simulates $\overline{M_A}$ on y. If $\overline{M_A}$ accepts y then N assumes the reply of the oracle is "no" and continues to simulate M on x. However if both M_A and $\overline{M_A}$ reject y then N rejects the input string x.

We now show that N correctly decides if $x \in L$. It is easy to note that if for any oracle query sring y, either M_A or $\overline{M_A}$ can accept y but not both. Also if M_A accepts y then $y \in A$. Similarly if $\overline{M_A}$ accepts y then $y \in \overline{A}$. As a result when either of these two NL machines accept then this can be taken to be the output of the oracle and N will proceed with its simulation of M. On the contrary if neither M_A nor $\overline{M_A}$ accept since NL is closed under complement we know that there exists at least one accepting computation path in exactly one of these Turing machines and we did not simluate both the Turing machines along any such

path. Therefore N also rejects x and stops the computation. Since N only simulates NL Turing machines it shows that N is also a NL machine. This shows that $\mathrm{NL}^{\mathrm{NL}} = \mathrm{NL}$. □

THEOREM 1.26. *Let* 2SAT=$\{\phi | \phi$ *is a Boolean formula in the 2-CNF such that* ϕ *is satisfiable*$\}$. *Then* 2SAT *is in* NL *and* $\overline{\text{2SAT}}$ *is in* NL

PROOF. We want to show that the language of all unsatisfiable Boolean formulas in the 2-CNF, denoted by $\overline{\text{2SAT}}$, is in NL. Let $\phi = (\phi_1 \wedge \phi_2 \wedge \cdots \wedge \phi_m)$ be in 2-CNF. We assume without loss of generality that each ϕ_i is a conjunction of two literals, where $1 \leq i \leq m$. Otherwise if there exists a clause ϕ_i that contains only one literal then we convert such a clause ϕ_i of ϕ into a conjunction of two new clauses where the first of the new clauses is a disjunction of the literal that occurs in ϕ_i along with a new variable y which is not in the 2-CNF Boolean formula ϕ given as input and the second new clause is the disjunction of the literal that occurs in ϕ_i and $\neg y$. It is easy to observe that the resulting 2CNF Boolean formula is satisfiable if and only if ϕ is satisfiable. Note that we use only one new variable y that is not in the 2-CNF Boolean formula ϕ given as input to convert all clauses that have only one literal into clauses that are the disjunction of two literals.

Now given a 2-CNF Boolean formula ϕ we define a directed graph $G = (V, E)$ as follows. For each variable x that occurs in ϕ we define two vertices in G. More precisely, we define a vertex for the variable x and a vertex for $\neg x$. Clearly if there are n variables in ϕ then there are $2n$ vertices in G. If $\phi_i = (\neg x \vee y)$ is a clause then we include directed edges (x, y) and $(\neg y, \neg x)$ in E of G. In other words, if we have a clause of ϕ to be $(\neg x \vee y)$ which is $(x \Rightarrow y)$ then we include the directed edge (x, y) in E. By commutativity, since $(\neg x \vee y)$ is also $(y \vee \neg x)$ which is $(\neg y \Rightarrow \neg x)$ we also include the edge $(\neg y, \neg x)$ in E of G. Clearly if there are m clauses in ϕ then there are $2m$ directed edges in G. Also it is easy to observe that there exists a directed path from a vertex α to a vertex β in G if and only if then there exists a directed path from $\neg\beta$ to $\neg\alpha$ in G. The 2-CNF Boolean formula ϕ is the conjunction of implications of the form $(\alpha \Rightarrow \beta)$, where α and β are literals in the clauses of ϕ. Given the conjunction of two clauses $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \gamma)$ we can simplify this conjunction and derive the clause $(\alpha \Rightarrow \gamma)$ from them. Using this we infer that the 2-CNF Boolean formula ϕ is unsatisfiable if and only if we can obtain a contradiction which is that there exists at least one variable x such that we can simplify the conjunction of clauses in ϕ to obtain the clause $(x \Rightarrow \neg x)$ and also that $(\neg x \Rightarrow x)$. However this is equivalent to the existence of edges in E of G such that there is a directed path from x to $\neg x$ and a directed path from $\neg x$ to x.

Now given a 2-CNF Boolean formula ϕ such that n is the size of ϕ, a $O(\log n)$-space bounded deterministic Turing machine can output the directed graph G described above. We then use a NL oracle to query if there exists a directed path from a variable x to $\neg x$ and also from $\neg x$ to x in G for all the variables x that occur in ϕ. If for some variable x we get the oracle reply to be "YES" for the queries submitted then we accept ϕ and output that ϕ is unsatisfiable. Otherwise we reject the input ϕ and output that ϕ is satisfiable. Now it follows from Theorem 1.25 that $\overline{\text{2SAT}}$ is in NL. □

THEOREM 1.27. $\overline{\text{2SAT}}$ *is hard for* NL *under* $\leq_m^{\mathrm{L}}$ *reductions*

PROOF. We prove this statement by showing that DSTCON $\leq_m^{\mathrm{L}}$ $\overline{\text{2SAT}}$. Let $G = (V, E)$ be a directed graph and let (G, s, t) be the input instance of DSTCON. Let $n = |V|$. We construct a 2CNF Boolean formula ϕ from (G, s, t) such that there exists a directed path from s to t in G if and only if ϕ is not satisfiable. Let x be the Boolean variable corresponding to the vertex s in G and $\overline{x}$ be the literal corresponding to the vertex t.

For each of the remaining vertices v_i in G, let there be a Boolean variable x_i. For each edge $(v_i, v_j) \in E$ let us include the clause $(v_i \Rightarrow v_j)$ in ϕ. In other words, for each edge $(v_i, v_j) \in E$ we include the clause $(\overline{v_i} \vee v_j)$ in ϕ. Apart from these clauses we also include the conjunction of the clause (x) in ϕ. Since the clause (x) is not a disjunction of two distinct literals we introduce a new variable y and include the conjunction of the following 2CNF Boolean subformula $(x \vee y) \wedge (x \vee \neg y)$ in ϕ. Clearly a $O(\log n)$-space bounded deterministic Turing machine can output ϕ given (G, s, t) as input.

Now consider the case when there exists a directed path from s to t in G. Let the vertices that form the directed path be the following in sequence: $s, v_i, v_j, \ldots, v_k, t$, where $1 \leq i, j, k \leq n$. We claim that ϕ is not satisfiable. To prove this claim consider the case when $x = F$. Then ϕ is not satisfiable since we have clauses $(x \vee y) \wedge (x \vee \neg y)$ in ϕ. For the case when $x = T$ let us consider the 2CNF Boolean subformula of ϕ formed by the clauses due to the edges in a directed path from s to$t$ in G: $(\overline{x} \vee v_i) \wedge (\overline{v_i} \vee v_j) \wedge \cdots \wedge (\overline{v_k} \vee \overline{x})$. It is easy to see that the Boolean subformula $(\overline{x} \vee v_i) \wedge (\overline{v_i} \vee v_j) \wedge \cdots \wedge (\overline{v_k} \vee \overline{x})$ is not satisfiable and so ϕ is also not satisfiable. Conversely, assume that there does not exist any directed path from s to t in G. Let A be the subset of vertices in V that are reachable from s in G, B be the subset of vertices in V from which t is reachable and $C = V \setminus (A \cup B)$. Since there is no directed path from s to t in G it is easy to observe that the subsets of V of vertices in A, B and C are pairwise disjoint and there are no edges from A to $B \cup C$ and from $A \cup C$ to B in G. Now let us assign the truth value T to every variable in $A \cup C$ including the variable x and the truth value F to every variable in B including the literal $\overline{x}$. It is easy to verify that this truth value assignment satisfies ϕ. This shows that DSTCON $\leq_m^{\mathrm{L}}$ $\overline{\mathrm{2SAT}}$. □

THEOREM 1.28. 2SAT *is complete for* NL *under* $\leq_m^{\mathrm{L}}$ *reductions*

PROOF. This result follows from Theorem 1.26, Theorem 1.27 and Theorem 1.22. □

CONJECTURE: ♯2SAT is complete for ♯L under $\leq_m^{\mathrm{L}}$ reductions.

1.4.2. Nondeterministic space bounded complexity classes above NL.

DEFINITION 1.29. Let Σ be the input alphabet and $S(n) \in \Omega(\log n)$. We define the complexity class NSPACE$(S(n)) = \{L \subseteq \Sigma^* | \exists$ a $O(S(n))$ space bounded non-deterministic Turing machine M such that $L = L(M)\}$.

DEFINITION 1.30. Let Σ be the input alphabet and $S(n) \in \Omega(\log n)$. We define the complexity class co-NSPACE$(S(n)) = \{\overline{L} \subseteq \Sigma^* | L \in \mathrm{NSPACE}(S(n))\}$.

THEOREM 1.31. **(The Immerman-Szelepcsényi Theorem)** NSPACE$(S(n))$ = co-NSPACE$(S(n))$, *where* $S(n) \in \Omega(\log n)$.

PROOF. Let Σ be the input alphabet and let $L \subseteq \Sigma^*$ such that $L \in \mathrm{NSPACE}(S(n))$. Let M be the NSPACE$(S(n))$ Turing machine that accepts L. As in the proof of Lemma 1.8 we observe that given an input $x \in \Sigma^*$, all the configurations of $M(x)$ is computable by a determinisitic Turing machine using space atmost $O(S(n))$ and therefore the directed configuration graph of $M(x)$ can also be output by a $O(S(n))$ space bounded deterministic Turing machine. As in Theorem 1.14 we observe that the directed st-connectivity problem for the simple layered directed acyclic graph obtained from the directed configuration graph G of a NSPACE$(S(n))$ Turing machine $M(x)$ is a canonical complete problem for NSPACE$(S(n))$. Now following the algorithm in the proof of Theorem 1.22 we are able to decide if there does not exist a directed path from

a vertex s in layer 1 of the above graph G to the vertex t in layer n of G in NSPACE($S(n)$). This shows that co-NSPACE($S(n)$) $\subseteq$ NSPACE($S(n)$) from which the theorem follows. □

1.5. Logarithmic Space Bounded Counting classes

In this section we assume without loss of generality that the input alphabet $\Sigma = \{0, 1\}$. We informally say that a complexity class $\mathcal{C}$ is a logarithmic space bounded counting class if the criterion to decide if an input string $x \in \Sigma^*$ is in a language $L \in \mathcal{C}$ is based on the number of accepting computation paths and/or the number of rejecting computation paths of a $O(\log n)$ space bounded non-deterministic Turing machine M. We also call logarithmic space bounded counting classes that have access to oracles which are logarithmic space bounded counting classes as logarithmic space bounded counting classes. **It follows from this informal definition that** NL **is the first and fundamental logarithmic space bounded counting class.**

We recall Definition 1.12.

Definition 1.32. Let Σ be the input alphabet. The complexity class $\sharp$L is defined to be the class of functions $f : \Sigma^* \to \mathbb{Z}^+$ such that there exists a NL Turing machine M for which we have $f(x) = acc_M(x)$ where $acc_M(x)$ denotes the number of accepting computation paths of M on any input $x \in \Sigma^*$.

Definition 1.33. Let Σ be the input alphabet. The complexity class GapL is defined to be the class of functions $f : \Sigma^* \to \mathbb{Z}$ such that there exists a NL machine M for which we have $f(x) = acc_M(x) - rej_M(x)$ where $acc_M(x)$ and $rej_M(x)$ denote the number of accepting and the number of rejecting computation paths of M on any input $x \in \Sigma^*$ respectively. We also denote $(acc_M(x) - rej_M(x))$ by $gap_M(x)$.

Definition 1.34. A language L belongs to the logarithmic space bounded counting class PL if there exists a GapL function f such that $x \in L$ if and only if $f(x) > 0$.

Definition 1.35. A language L belongs to the logarithmic space bounded counting class C$_=$L if there exists a GapL function f such that $x \in L$ if and only if $f(x) = 0$.

We need the following result.

Proposition 1.36. (Folklore)

(1) *Let Σ be the input alphabet and $f : \Sigma^* \to \mathbb{Z}$ such that $f(x) \geq 0$, $\forall x \in \Sigma^*$ and $f \in$ FL. Then $f \in \sharp$L.*
(2) $\sharp$L *is closed under addition.*

Corollary 1.37. *Let Σ be the input alphabet and $L \subseteq \Sigma^*$ be a language in* NL*. Then $\exists f \in \sharp$L such that for any given input $x \in \Sigma^*$, we have $x \in L$ if and only if $f(x) \geq 2^{n^k}$, where 2^{n^k+1} is the number of computation paths of the* NL *Turing machine M of f, and $k \geq 0$ is a constant.*

Proof. Let M' be the NL Turing machine that accepts L such that on any input x of length n there are 2^{n^k} computation paths of M' on x. For any given input $x \in \Sigma^*$, we know that $x \in L$ if and only if $acc_{M'}(x) > 0$. Let $f(x) = acc_{M'}(x) + (2^{n^k} - 1)$, where $n = |x|$. It follows from Proposition 1.36 that $f \in \sharp$L. Also if M is the NL Turing machine of f then the number of computation paths of M on any input x of length n is 2^{n^k+1}. Clearly $x \in L$ if and only if $f(x) \geq 2^{n^k}$ which proves our result. □

Corollary 1.38. *Let Σ be the input alphabet and $L \subseteq \Sigma^*$ be a language in* NL*. Then $\exists g \in \sharp$L such that for any given input $x \in \Sigma^*$, we have $x \in L$ if and only if $g(x) > 2^{n^k}$, where 2^{n^k+1} is the number of computation paths of the* NL *Turing machine M of g, and $k \geq 0$ is a constant.*

COROLLARY 1.39. NL $\subseteq$ PL.

PROOF. Follows from Definition 1.34 and Corollary 1.38. $\square$

1.6. NL = $C_=L$

DEFINITION 1.40. The symmetric difference of two sets A and B is the set of elements that belong to exactly one of A and B. The symmetric difference of two sets A and B is denoted by $A \triangle B$. In other words, $A \triangle B = (\overline{A} \cap B) \cup (A \cap \overline{B})$.

DEFINITION 1.41. Let Σ be the input alphabet. We define (NL$\triangle$NL) = $\{L \subseteq \Sigma^* | \exists L_1, L_2 \in$ NL and $L = L_1 \triangle L_2$, where $L_1, L_2 \subseteq \Sigma^*\}$.

We now deduce the following result from Corollary 1.37 and Corollary 1.38.

THEOREM 1.42. $C_=L \subseteq$ (NL$\triangle$NL)

PROOF. Let Σ be the input alphabet, $L \in C_=L$ and let M be a NL Turing machine such that on any input $x \in \Sigma^*$, we have $x \in L$ if and only if $gap_M(x) = 0$. For any input $x \in \Sigma^*$ such that $n = |x|$, we assume without loss of generality that all the paths of the computation tree of M on input x have depth n^k, where $k > 0$ is a constant and which clearly depends on the $O(\log n)$ space used by $M(x)$ along any of its computation paths. As a result the number of computation paths of $M(x)$ is 2^{n^k}, and $x \in L$ if and only if $acc_M(x) = 2^{n^k - 1}$.

Consider the following. Let $L_1 = \{x \in \Sigma^* | acc_M(x) \geq 2^{n^k - 1}\}$ and let $L_2 = \{x \in \Sigma^* | acc_M(x) > 2^{n^k - 1}\}$. It follows from Corollary 1.37 and Corollary 1.38 that $L_1, L_2 \in$ NL. Also $L_2 \subseteq L_1$ and we also deduce that $L = L_1 \triangle L_2$. Thus $L \in$ NL$\triangle$NL which implies $C_=L \subseteq$ (NL$\triangle$NL). $\square$

THEOREM 1.43. (NL$\triangle$NL) $\subseteq L^{NL}$.

PROOF. Given two sets A and B it is easy to note that $(A \triangle B) = (\overline{A} \cap B) \cup (A \cap \overline{B})$. Now let $A, B \in$ NL. Due to Theorem 1.22 and Proposition 1.24, it follows that $(A \triangle B) \in$ NL. Clearly to determine if an input string $x \in \Sigma^*$ is in $(A \triangle B)$ a logarithmic space bounded deterministic Turing machine that has access to a NL oracle shall make one query and output if x is in $(A \triangle B)$ or not. This shows that (NL$\triangle$NL) $\subseteq L^{NL}$. $\square$

We therefore obtain the following result from Theorem 1.42, Theorem 1.43 and Theorem 1.23.

COROLLARY 1.44. $C_=L \subseteq$ NL.

LEMMA 1.45. NL $\subseteq$ co-$C_=L$.

PROOF. Let $L \subseteq \Sigma^*$ such that $L \in$ NL and let M be the NL-Turing machine which accepts L. Let $f \in \sharp L$ such that on any input $x \in \Sigma^*$ we have $f(x) = acc_M(x)$ $\forall x \in \Sigma^*$. For any input $x \in \Sigma^*$ such that $n = |x|$, we assume without loss of generality that all the paths of the computation tree of M on input x have depth n^k, where $k > 0$ is a constant and which clearly depends on the $O(\log n)$ space used by $M(x)$ along any of its computation paths. Let us consider the following algorithm which can be implemented by a NL-Turing machine, say M', on a given input $x \in \Sigma^*$, where $n = |x|$:

Non-deterministically choose to either
{simulate M on input x} or
{form a computation tree having depth n^k and accept the input x on all of these computation paths}.

On any given input $x \in \Sigma^*$, if $x \in L$ then $f(x) \geq 1$. As a result $gap_{M'}(x) = acc_{M'}(x) - rej_{M'}(x) > 0$. However if $x \notin L$ then all the computation paths of M on input x end in the rejecting configuration and therefore $acc_{M'}(x) = rej_{M'}(x)$ which clearly implies $gap_{M'}(x) = 0$. This shows that $L \in$ co-C$_=$L from which we get NL $\subseteq$ co-C$_=$L. □

THEOREM 1.46. NL $\subseteq$ C$_=$L.

PROOF. From Lemma 1.45 we infer that co-NL $\subseteq$ C$_=$L. Also as a consequence of Theorem 1.22 we obtain that NL $\subseteq$ C$_=$L. □

COROLLARY 1.47. NL = C$_=$L.

1.7. The Isolating Lemma

Let $A = \{a_1, a_2, \ldots, a_m\}$ be a set of elements and let $\mathcal{F}$ be a collection of non-empty subsets of A. Let W be a finite number of consecutive integers, which are called as integer weights, and assume that we shall assign integer weights to elements of A using a function $f : A \to W$. Define the weight of a subset S of A, denoted by $f(S)$, to be the sum of the weights of the elements in the subset S.

We say that a weight function f *is good for* $\mathcal{F}$ if there is exactly one minimum-weight set in $\mathcal{F}$ with respect to f and we say that f *is bad for* $\mathcal{F}$ otherwise.

THEOREM 1.48. **(The Isolating Lemma)** *Let A be a finite set of cardinality m and let $\mathcal{F}$ be a family of non-empty subsets of A. Let $r > 2m$ and let f be a function $f : A \to \{1, \ldots, r\}$ that assigns a value independently and uniformly at random to each element in $a_i \in A$ from the set $\{1, \ldots, r\}$, where $1 \leq i \leq m$. Then,*

$$Pr[f \text{ is good for } \mathcal{F}] \geq \frac{1}{2}.$$

PROOF. Let $MinWeight_f(\mathcal{F}) = \min\{f(S) | S \in \mathcal{F}\}$ and let $MinWeightSet_f(\mathcal{F}) = \{S \in \mathcal{F} | f(S) = MinWeight_f(\mathcal{F})\}$. Recall from the definition of good and bad functions stated above that any weight function f is bad for $\mathcal{F}$ if and only if $|MinWeightSet_f(\mathcal{F})| \geq 2$.

For $x \in A$, we say that the minimum-weight sets of $\mathcal{F}$ with respect to f are unambiguous about the inclusion of x if there exists $S, S' \in MinWeightSet_f(\mathcal{F})$ such that $x \in (S \setminus S') \cup (S' \setminus S)$. It is easy to note that f is bad for $\mathcal{F}$ if and only if there exists some $x \in A$ such that the minimum-weight sets of $\mathcal{F}$ with respect to f are ambiguous about the inclusion of x.

Now assume that the function f is bad for $\mathcal{F}$. In other words, the minimum weight sets of $\mathcal{F}$ with respect to f are ambiguous about the inclusion of x, for some $x \in A$. Let $f' : A \to \{1, \ldots, r\}$ be another function such that for any $S \subseteq A$, if $x \notin S$ then $f'(S) = f(S)$. However if $x \in S$ then $f'(S) \neq f(S)$ otherwise.

Claim. Minimum-weight sets of $\mathcal{F}$ with respect to f' are unambiguous about the inclusion of x.

Proof of Claim. To prove this, let $\delta(x) = f'(x) - f(x)$. Suppose that $\delta(x) > 0$. For any $S \subseteq A$, if $x \in S$ then $f'(S) = f(S) + \delta(x)$ and $f'(S) = f(S)$ otherwise. This implies that $MinWeight_f(\mathcal{F}) = MinWeight_{f'}(\mathcal{F})$ and $MinWeightSet_{f'}(\mathcal{F}) = \{S \in MinWeightSet_f(\mathcal{F}) | x \notin S\}$. Next suppose that $\delta(x) < 0$. Then, for all $S \in \mathcal{F}$, we have $f'(S) = f(S) - |\delta(x)|$ if $x \in S$ and $f'(S) = f(S)$ otherwise. This implies that $MinWeight_f(\mathcal{F}) = MinWeight_{f'}(\mathcal{F}) - |\delta(x)|$ and $MinWeightSet_{f'}(\mathcal{F}) = \{S \in MinWeightSet_f(\mathcal{F}) | x \in S\}$. Therefore, if $\delta(x) > 0$ then all the minimum-weight sets of $\mathcal{F}$ with respect to f' contain x and if $\delta(x) < 0$ then there does not exist any minimum-weight set of $\mathcal{F}$ with respect to f' that contains x. This proves our claim.

Now if there exists some $x \in A$ such that the minimum-weight sets of $\mathcal{F}$ with respect to f are ambiguous about the inclusion of x, then certainly there exists an element $y \in A$ such that $y \neq x$ and the minimum-weight sets of $\mathcal{F}$ with respect to f are ambiguous about the inclusion of y. If the number of such elements in A is exactly 2, then the number of functions f' that we can obtain from f as in the claim stated above such that these two minimum-weight sets of $\mathcal{F}$ with respect to f' are unambiguous about the inclusion of x, y is atmost r^{m-2}. Since there can be $\binom{m}{2}$ such pairs x, y, the number of such functions $f : A \to \{1, \ldots, r\}$ is atmost $\binom{m}{2} r^{m-2}$. Note that not all of these functions have to be bad for $\mathcal{F}$. Extending this observation, if there were k many such elements in A such that the minimum-weight sets of $\mathcal{F}$ with respect to f are ambiguous about the inclusion of any of these elements, then the number of functions that we can obtain from f as done in the claim stated above is atmost $\binom{m}{k} r^{m-k}$. However this quantity is $\leq mr^{m-1}$. As a result, the proportion of functions f such that f is bad for $\mathcal{F}$ is $\leq \frac{mr^{m-1}}{r^m} \leq \frac{1}{2}$. Therefore the proportion of functions f that are good for $\mathcal{F}$ is $\geq \frac{1}{2}$ and this proves our theorem. □

We recall the definition of DSTCON from Definition 1.7.

COROLLARY 1.49. *Let $G = (V, E)$ be a directed graph which does not have any self-loops such that $s, t \in V$ and let (G, s, t) be a "yes" input instance of* DSTCON. *Let us consider the function $f : E \to \{1, \ldots, 4|E|\}$ that assigns a value independently and uniformly at random to every edge in E of G. Then,*

$$Pr[\textit{there exists a directed path from } s \textit{ to } t \textit{ in } G \textit{ that has unique minimum weight}] \geq \frac{3}{4},$$

where the weight of a directed path is the sum of the weights of the edges in the directed path.

PROOF. We obtain this result by following the proof of Theorem 1.48. □

1.8. Uniqueness of directed paths in weighted directed graphs

THEOREM 1.50. *Let A be a finite set of cardinality m and let $\mathcal{F}_1, \mathcal{F}_2, \ldots, \mathcal{F}_n$ be families of non-empty subsets of A, where $n \leq m^k$ for some $k \geq 0$. Also let $r > \max(4m, 4n)$. Then there exists a function $f : A \to \{1, \ldots, r\}$ such that f is good for every $\mathcal{F}_i$, where $1 \leq i \leq n$.*

PROOF. We show this result using the following simple counting method, known as the Adleman's technique. Form a $0, 1$ matrix M of dimension $n \times r^n$ with n rows and r^n columns, where the i^{th} row of M corresponds to the family $\mathcal{F}_i$ and the j^{th} column corresponds to the function f_j in the lexicographically increasing order, for $1 \leq i \leq n$ and $1 \leq j \leq r^n$. (Here we assume that every function f is considered as a vector, and given two functions f and g we say that f is lexicographically lesser than g, denoted by $f \prec g$, if f is lesser than g when compared component-wise as vectors). We put $M_{ij} = 1$ if the function f_j is good for the family $\mathcal{F}_i$ and $M_{ij} = 0$ otherwise. By the choice of r and from the proof of Theorem 1.48, it follows that

$$\Pr[f \textit{ is good for } \mathcal{F}_i] \geq \frac{3}{4},$$

where f is a function chosen independently and uniformly at random, and $1 \leq i \leq n$. Now using induction on m, it is easy to see that there exists a column in M that contains only 1. The function corresponding to this column satisfies the condition stated in this theorem. □

COROLLARY 1.51. *Let $G = (V, E)$ be a directed graph which does not have any self-loops and let $m = |E|$. Let n denote the number of ordered pairs of vertices (s, t) such that there exists a directed path from s to t in G. Also let $r > \max(4m, 4n)$. Then, we have a function $f : E \to \{1, \ldots, r\}$ that assigns weights to the*

edges of G, such that there exists a directed path of unique minimum weight between every pair of vertices (s,t) for which there exists a directed path from s to t in G, where the weight of a directed path is the sum of the weights of the edges in the directed path.

PROOF. Given a directed graph $G = (V, E)$, the number of pairs of vertices between which there exists a directed path, denoted by n, is $\leq 2\binom{v}{2}$, where $v = |V|$. If $s, t \in V$ is such that there exists a directed path from s to t in G, then the set of edges corresponding to every directed path from s to t is a subset of E. Considering all directed paths from s to t we get the family of non-empty subsets of E, which we denote by $\mathcal{F}$. As a result there are n families $\mathcal{F}_1, \ldots, \mathcal{F}_n$. Also let $m = |E|$. Now apply Theorem 1.50 to obtain the function f and this completes the proof. □

1.9. Min-unique graphs and Unambiguous Logarithmic space: UL

DEFINITION 1.52. A min-unique graph is a directed graph with positive weights associated with each edge where for every pair of vertices u, v, if there is a path from u to v, then there is a unique minimum weight path from u to v. Here, the weight of a path is the sum of the weights on its edges.

We now define the complexity class unambiguous logarithmic space, UL.

DEFINITION 1.53. Let Σ be the input alphabet. We say that $L \subseteq \Sigma^*$ is a language in the complexity class UL if there exists a function $f \in \sharp\mathrm{L}$ such that for any input $x \in \Sigma^*$ we have $f(x) = 1$ if $x \in L$ and $f(x) = 0$ if $x \notin L$.

We now define co-UL, based on the definition of co-$\mathcal{C}$ from Definition 1.1 where $\mathcal{C}$ is a complexity class.

DEFINITION 1.54. Let Σ be the input alphabet. For a language $L \in \Sigma^*$, the complement of L is $\overline{L} = \Sigma^* - L$. We define the complexity class co-UL $= \{\overline{L} \subseteq \Sigma^* | L \in \mathrm{UL}\}$.

DEFINITION 1.55. Let Σ be the input alphabet. We say that $L \in \mathrm{UL} \cap \text{co-UL}$ if $L \in \mathrm{UL}$ and $L \in \text{co-UL}$.

THEOREM 1.56. *Let $\mathcal{G}$ be a class of graphs and let $H = (V, E) \in \mathcal{G}$. If there is a polynomially bounded logarithmic space computable function f that on input H outputs a weighted graph $f(H)$ so that*

(1) *$f(H)$ is min-unique, and*
(2) *H has an st-path if and only if $f(H)$ has an st-path*

then the st-connectivity problem for $\mathcal{G}$ is in $\mathrm{UL} \cap \text{co-UL}$.

PROOF. It suffices to give a $\mathrm{UL} \cap \text{co-UL}$ algorithm for the reduced graph. For $H \in \mathcal{G}$, let $G = f(H)$ be a directed graph with a min-unique weight function w on its edges. We first construct an unweighted graph G' from G by replacing every edge e in G with a path of length $w(e)$. It is easy to see that st-connectivity is preserved. That is, there is an st-path in G if and only if there is one in G'. Since G is min-unique, it is straightforward to argue that the shortest path between any two vertices in G' is unique.

Let c_k and Σ_k denote the number of vertices which are at a distance at most k from s and the sum of the lengths of the shortest path to each of them, respectively. Let $d(v)$ denote the length of the shortest path from S to v. If no such path exists, then $d(v) = |V| + 1$. We have,

$$\Sigma_k = \sum_{\{v \in V \,|\, d(v) \leq k\}} d(v).$$

Algorithm 3 Shortest-Path: Determining whether $d(v) \leq k$ or not
Input: (G, v, k, c_k, Σ_k)
Output: *true* if $d(v) \leq k$ else *false*

```
1:  Initialize count ← 0; sum ← 0; path.to.v ← false
2:  for each x ∈ V do
3:      Non-deterministically guess if d(x) ≤ k
4:      if guess is Yes then
5:          Guess a path of length l ≤ k from s to x
6:          if guess is correct then
7:              Set count ← count + 1
8:              Set sum ← sum + l
9:              if x = v then
10:                 Set path.to.v ← true
11:         else return "?"
12: if count = c_k and sum = Σ_k then
13:     return path.to.v
14: else
15:     return "?"
```

We first give an unambiguous routine (Algorithm 3) to evaluate the predicate "$d(v) \leq k$" when given the values of c_k and Σ_k. The algorithm will output the correct value of the predicate (*true/false*) on a unique path and outputs "?" on the rest of the paths.

We will argue that Algorithm 3 is unambiguous.

(1) If Algorithm 3 incorrectly guesses that $d(x) > k$ for some vertex x then $count < c_k$ and so it returns "?" in line 18. Thus, consider the computation paths that correctly guess the set $\{x | d(x) \leq k\}$.

(2) If at any point the algorithm incorrectly guesses the length l of the shortest path to x, then one of the following two cases occur.

   (a) If $d(x) > l$ then no path from s to x would be found and the algorithm returns "?" in line 11.

   (b) If $d(x) < l$ then the variable *sum* would be incremented by a value greater than $d(x)$ and thus *sum* would be greater than Σ_k causing the algorithm to return "?" in line 18.

Thus there will remain only one computation path where all the guesses are correct and the algorithm will output the correct value of the predicate on this unique path. Finally, we note that Algorithm 3 is easily seen to be computable in logarithmic space.

Algorithm 4 Computing c_k and Σ_k
Input: $(G, k, c_{k-1}, \Sigma_{k-1})$
Output: c_k, Σ_k

```
1: Initialize c_k ← c_{k-1} and Σ_k ← Σ_{k-1}
2: for each x ∈ V do
3:     if ¬(d(v) ≤ k − 1) then
4:         for each x such that (x, v) ∈ E do
5:             if d(x) ≤ k − 1 then
6:                 Set c_k ← c_k + 1
7:                 Set Σ_k ← Σ_k + k
8: return c_k and Σ_k
```

Algorithm 5 Determining if there exists a path from s to t in a min-unique graph G
Input: A min-unique graph G
Output: *true* if there exists a directed path from s to t; *false* otherwise

1: Initialize $c_0 \leftarrow 1, \Sigma_0 \leftarrow 0, k \leftarrow 0$
2: **for** $k \leftarrow 1, \ldots n$ **do**
3: Compute c_k and Σ_k by invoking Algorithm 4 on $(G, k, c_{k-1}, \Sigma_{k-1})$
4: Invoke Algorithm 3 on (G, t, n, c_n, Σ_n) and return its value

Next, we describe an unambiguous procedure (Algorithm 4) that computes c_k and Σ_k given c_{k-1} and Σ_{k-1}. Algorithm 4 uses Algorithm 3 as a subroutine. Other than calls to Algorithm 3, this routine is deterministic, and so it follows that Algorithm 4 is also unambiguous.

We will argue that Algorithm 4 computes c_k and Σ_k. The subgraph consisting only of s ($d(x) \leq 0$) is trivially min-unique and $c_0 = 1$ and $\Sigma_0 = 0$. Inductively, it is easy to see that

$$c_k = c_{k-1} + |\{v | d(v) = k\}|,$$
$$\Sigma_k = \Sigma_{k-1} + k|\{v | d(v) = k\}|.$$

In addition, $d(v) = k$ if and only if there exists $(x, v) \in E$ such that $d(x) \leq k-1$ and $\neg(d(v) \leq k-1)$. Both of these predicates can be computed using Algorithm 3. Combining these facts, we see that Algorithm 4 computes c_k and Σ_k given c_{k-1} and Σ_{k-1}.

As a final step, we give the main routine that invokes Algorithm 4 to check for st-connectivity in a min-unique graph. Since there is an st-path if and only if $d(t) \leq n$, it suffices to compute c_n and Σ_n and invoke Algorithm 3 on (G, t, n, c_n, Σ_n). This procedure is presented as Algorithm 5. To ensure that the algorithm runs in logarithmic space, we do not store all intermediate values for c_k, Σ_k. Instead, we only keep the most recently computed values and re-use space. As with Algorithm 4, this procedure is deterministic and so the entire routine is unambiguous. Thus reachability in min-unique graphs can in fact be decided in UL $\cap$ co-UL. □

1.10. NL/poly = (UL $\cap$ co-UL)/poly

Definition 1.57. Let $\Sigma = \{0, 1\}$ be the input alphabet and let $\mathcal{C}$ be a complexity class. We define $\mathcal{C}/poly$ as the complexity class of all languages $L \subseteq \Sigma^*$for which there exists a sequence of "advice strings" $\{\alpha(n) | n \in \mathbb{N}\}$ and a language $L' \in \mathcal{C}$ such that $x \in L$ if and only if $(x, \alpha(x)) \in L'$, where $x \in \Sigma^*$.

Theorem 1.58. (UL $\cap$ co-UL)/poly $\subseteq$ NL/poly.

Proof. It is easy to see from Definition 1.53 and Definition 1.2 that UL $\subseteq$ NL. Also we know from the Immerman-Szelepscényi Theorem (shown in Theorem 1.22) that NL = co-NL and so (UL $\cap$ co-UL) $\subseteq$ NL. Using Definition 1.57, it is now trivial to show that (UL $\cap$ co-UL)/poly $\subseteq$ NL/poly. □

Theorem 1.59. NL/poly $\subseteq$ (UL $\cap$ co-UL)/poly.

Proof. Using Definition 1.57, we infer that it is sufficient to show that NL $\subseteq$ (UL $\cap$ co-UL)/poly. We know from Theorem 1.10 that, DSTCON is complete for NL under $\leq_m^{\mathrm{L}}$. Therefore to prove our result, it is sufficient to show that DSTCON $\in$ (UL $\cap$ co-UL)/poly.

Let us now consider an input instance (G, s, t) of DSTCON. We assume without loss of generality that G does not have any self-loops. It follows from Corollary 1.51 that, we have a function $f : E \to \{1, \ldots, r\}$

that assigns weights to the edges of G, such that there exists a directed path of unique minimum weight between every pair of vertices (s, t) for which there exists a directed path from s to t in G, where the weight of a directed path is the sum of the weights of the edges in the directed path.

Assume we are given an input instance (G, s, t) of DSTCON. Let the function stated in Corollary 1.51 be given as an advice string which is a vector of weights $\vec{\mathbf{w}} = (w_1, w_2, \ldots, w_m)$, where $m = |E|$. Clearly $\vec{\mathbf{w}}$ is of length which is polynomial in the size of (G, s, t). We observe that upon assigning weights to the edges of G according to $\vec{\mathbf{w}}$, the input directed graph G is min-unique. Now using this advice string and Theorem 1.56, it follows that we can decide if there exists a directed path from s to t in G in $(\text{UL} \cap \text{co-UL})$. As a result it follows that DSTCON $\in (\text{UL} \cap \text{co-UL})/\text{poly}$ which proves our result. □

THEOREM 1.60. $\text{NL}/\text{poly} = (\text{UL} \cap \text{co-UL})/\text{poly}$.

CHAPTER 2

Modulo-based and Probabilistic Logarithmic Space Bounded Counting Classes

2.1. Salient properties of GapL

LEMMA 2.1. *Let Σ be the input alphabet. If $f(x)$ is a $\sharp$L or* GapL *function then there exists a polynomial $p(n)$ such that the absolute value of $f(x)$ is bounded above by $2^{p(n)}$ for every $x \in \Sigma^*$, where $n = |x|$.*

PROOF. Either $f(x) = acc_M(x)$ or $gap_M(x)$ for some NL Turing machine M. Let $s(n) \in O(\log n)$ be an upper bound the amount of space that M uses on input strings of length n. It is easy to note that the running time of $M(x)$ is at most $p(|x|) = 2^{s(|x|)}$, which is a polynomial in $|x|$. As a result the number of distinct computation paths of M on any input x is upper bounded by $2^{p(|x|)}$ from which it follows that the number of accepting and rejecting computation paths of M on any input x is at most $2^{p(|x|)}$ and this proves our claim. □

LEMMA 2.2. *If $f \in$* GapL *then $-f \in$* GapL*.*

LEMMA 2.3. *Let Σ be the input alphabet. For every* NL *Turing machine M, there is a* NL *machine N such that $gap_N(x) = acc_M(x)$ for every input $x \in \Sigma^*$.*

PROOF. Given an input x, the Turing machine N guesses a path p of $M(x)$. If p is accepting, N accepts. Otherwise, N branches once, accepting on one branch and rejecting on the other. We have, for all x, $gap_N(x) = acc_N(x) - acc_{\overline{N}}(x) = acc_N(x) - acc_{\overline{M}}(x) = (acc_M(x) + acc_{\overline{M}}(x)) - acc_{\overline{M}}(x) = acc_M(x)$. □

COROLLARY 2.4. $\sharp$L $\subset$ GapL.

PROOF. Follows from Lemma 2.3 and the fact that GapL contains functions that take negative values on many inputs. □

Note that a $\sharp$L function can take only nonnegative values and so this class cannot capture all the functions in FL that evaluate to integer values on any given input. However, it does capture those functions in FL that only take nonnegative integer values on all inputs. The class GapL encompasses any function in FL that takes integer values on any input without any restriction. For the rest of this chapter, we assume without loss of generality that any function $f \in$ FL that we consider always takes integer values on any input.

THEOREM 2.5. *If an* FL *function f always takes on nonnegative values then f is in $\sharp$*L*. Every* FL *function f is in* GapL*.*

PROOF. Let f be a FL function that always takes non-negative values. Define a NL Turing machine M, that on input x guesses $f(x)$ computation paths and accepts on each of these paths.. Clearly $acc_M(x) = f(x)$ when $f(x) \geq 0$, for all $x \in \Sigma^*$, where Σ is the input alphabet.

Now let $f \in \mathrm{FL}$. Define a NL Turing machine N which on input x computes $f(x)$. For any $f(x)$ whether it is positive or negative, it follows from Corollary 2.4 that there exists a NL Turing machine N' such that $gap_{N'}(x) = |f(x)|$. Therefore after computing $f(x)$, if $f(x) > 0$ then N simulates N' on input x. However, if $f(x) < 0$, then N simulates N' on input x and rejects along the computation paths in which N' accepts and accepts along the computation paths in which N' rejects. This shows that $gap_N(x) = f(x)$. □

LEMMA 2.6. *For all functions f, the following are equivalent:*

(1) $f \in \mathrm{GapL}$.
(2) *f is the difference of two $\sharp\mathrm{L}$ functions.*
(3) *f is the difference of a $\sharp\mathrm{L}$ and* FL *function.*
(4) *f is the difference of a* FL *function and a* GapL *function.*

PROOF. (1 ⇒ 2) For any M we have $gap_M(x) = acc_M(x) - acc_{\overline{M}}(x)$ by definition, so GapL is the difference of two $\sharp\mathrm{L}$ functions.

(2 ⇒ 3) Let f and g be $\sharp\mathrm{L}$ functions. We can assume that $f = acc_M$ and $g = acc_N$, where M and N are NL Turing machines, and N has $2^{q(n)}$ computation paths for some polynomial q (just pad N with extra rejecting computation paths so that all computation paths have length $q(n)$). Let M' be the NL Turing machine that first branches once, then simulates M on one branch and $\overline{N}$ on the other. We have, for any x, $f(x) - g(x) = acc_M(x) - acc_N(x) = acc_M(x) + acc_{\overline{N}}(x) - 2^{q(|x|)} = acc_{M'}(x) - 2^{q(|x|)}$. Therefore $f - g$ is the difference of a $\sharp\mathrm{L}$ and a FL function.

(3 ⇒ 1) Let f be a $\sharp\mathrm{L}$ function and let $g \in \mathrm{FL}$. Let M be the NL Turing machine such that $f = gap_M(x)$. Let N be such that, for all $x \in \Sigma^*$, $N(x)$ resembles $M(x)$ padded with $g(x)$ rejecting computation paths. We then have $gap_N(x) = f(x) - g(x)$.

(3 ⇔ 4) This follows since we now have (1 ⇔ 3), and the GapL functions are closed under negation. □

COROLLARY 2.7. $\mathrm{GapL} \subseteq \mathrm{FL}^{\sharp\mathrm{L}[1]}$.

PROOF. Follows from the third characterization of GapL in the above lemma. □

The composition of two FL functions remains a FL function. However it seems unlikely that the composition of two $\sharp\mathrm{L}$ functions is a $\sharp\mathrm{L}$ or even a GapL function. We can however, combine $\sharp\mathrm{L}$ and GapL functions with FL functions.

LEMMA 2.8. *Let f be a* FL *function and g a $\sharp\mathrm{L}$ or* GapL *function. Then $g(f(x))$ is a $\sharp\mathrm{L}$ or* GapL *function, respectively.*

PROOF. Let M be a NL Turing machine that defines a $\sharp\mathrm{L}$ function g. Define N to be a NL Turing machine that on input x simulates $M(f(x))$. Then $acc_N(x)$ is exactly $g(f(x))$. The proof for GapL is identical. □

LEMMA 2.9. (1) *Let Σ be the input alphabet, $f \in \mathrm{GapL}$ and $p(n)$ be a polynomial. Let $h : \Sigma^* \to \mathbb{Z}$ be defined for all $x \in \Sigma^*$ by $h(x) = \Sigma_{1 \le i \le p(|x|)} f(\langle x, i \rangle)$. Then $h \in \mathrm{GapL}$.*

(2) *Let Σ be the input alphabet, $f \in \sharp\mathrm{L}$ and $p(n)$ be a polynomial. Let $h : \Sigma^* \to \mathbb{Z}$ be defined for all $x \in \Sigma^*$ by $h(x) = \Sigma_{1 \le i \le p(|x|)} f(\langle x, i \rangle)$. Then $h \in \sharp\mathrm{L}$.*

PROOF. (1) Let $f = gap_M(x)$ for some NL Turing machine M. Define N to be the NL Turing machine that on input $x \in \Sigma^*$, guesses a positive integer i between 1 and $p(|x|)$ and simulates

a path of M on the input $\langle x, i\rangle$. N accepts the input $\langle x, i\rangle$ if M accepts and rejects the input otherwise. Then, for every $x \in \Sigma^*$, $gap_N(x) = h(x)$.

(2) The proof is analogous to the above case when $f \in \text{GapL}$.

□

LEMMA 2.10. *Let Σ be the input alphabet, $f \in \text{GapL}$ and $p(n)$ be a polynomial. Let $h : \Sigma^* \to \mathbb{Z}$ be defined for all $x \in \Sigma^*$ by $h(x) = \prod_{1 \le i \le p(|x|)} f(\langle x, i\rangle)$. Then $h \in \text{GapL}$.*

PROOF. Let $f = gap_M$ for some NL Turing machine M. For each $x \in \Sigma^*$ and i such that $1 \le i \le p(|x|)$, let $S(x, i)$ denote the set of all computation paths of x on input $\langle x, i\rangle$ and, furthermore, for each $\pi \in S(x, i)$, define $\alpha(x, i, \pi) = 1$ if π is an accepting computation path and it is -1 otherwise. Then, for each $x \in \Sigma^*$, $h(x) = \Sigma_{\pi \in S(x,1)} \cdots \Sigma_{\pi \in S(x,1)} \alpha(x, 1, \pi_1) \cdots \alpha(x, p(|x|), \pi_{p(|x|)})$.

Define N to be the NL Turing machine that on input $x \in \Sigma^*$ behaves as follows: N non-deterministically guesses and simulates a path of M on input $\langle x, i\rangle$, for all $1 \le i \le p(|x|)$. In the course of doing this, N computes the parity of the number of values of i, $1 \le i \le p(|x|)$, such that M on input $\langle x, i\rangle$ rejects. When all the simulations have been completed, N accepts x on its current computation path if and only if this value is even. Note that, for every $x \in \Sigma^*$, on the path of N on input x corresponding to the guesses $(\pi_1, \pi_2, \ldots, \pi_{p(|x|)})$, the product $\alpha(x, 1, \pi_1) \cdots \alpha(x, p(|x|), \pi_{p(|x|)})$ is 1 if and only if N accepts along this computation path and that the product is -1 if and only if N rejects along this computation path. Thus for every $x \in \Sigma^*$, we have $gap_N(x) = h(x)$. Since p is a polynomial and M is a NL Turing machine, N is also a NL Turing machine. Thus $h \in \text{GapL}$. □

2.2. A logarithmic space bounded modulo counting class: ModL

DEFINITION 2.11. Let Σ be the input alphabet. A language $L \subseteq \Sigma^*$ is in the complexity class ModL if there is a function $f \in \text{GapL}$ and a function $g \in \text{FL}$ such that on any input $x \in \Sigma^*$,

- $g(x) = 1^{p^e}$ for some prime p and a positive integer e, and
- $x \in L \Leftrightarrow f(x) \not\equiv 0 (\text{mod } p^e)$.

LEMMA 2.12. $\text{FL}^{\text{ModL}} = \text{FL}^{\text{GapL}}$.

PROOF. For the forward inclusion it suffices to show that $\text{ModL} \subseteq \text{L}^{\text{GapL}}$. Suppose $L \in \text{ModL}$ is witnessed by a function $f \in \text{GapL}$ and a function $g \in \text{FL}$ that computes prime powers in unary. Now, a logarithmic space bounded deterministic Turing machine can retrieve the value of $f(x)$ for any query x using the GapL oracle. It is also easy to see that checking if $f(x) \not\equiv 0 (\text{mod } |g(x)|)$ is also computable in logarithmic space. As a result it follows that $L \in \text{L}^{\text{GapL}}$. Therefore $\text{L}^{\text{ModL}} \subseteq \text{L}^{\text{GapL}}$.

For the reverse inclusion, let $L \in \text{L}^{\text{GapL}}$ be computed by a logarithmic space bounded deterministic Turing machine with access to a function $f \in \text{GapL}$ as an oracle. (Due to Corollary 2.7, since $\text{L}^{\sharp\text{L}} = \text{L}^{\text{GapL}}$ we can assume that the function f is non-negative on all inputs). For $x \in \Sigma^n$, we have $\text{size}(f(x)) \le p(n)$, for some polynomial $p(n)$. By the Prime Number Theorem, the number of primes between 2 and $p^2(n)$ is $p^2(n)/O(l \log n) > p(n)$, for sufficiently large n. Thus the first $p(n)$ primes are of size $O(\log n)$. Furthermore, the product of the first $p(n)$ primes is greater than $f(x)$. Also it is easy to see that checking if a $O(\log n)$-bit integer is a prime can be done in logarithmic space. Furthermore, in logarithmic space, we can also compute the i^{th} prime for $1 \le i \le p(n)$. Let p_i denote the i^{th} prime. We define the function $g \in \text{FL}$ as follows: $g(x, 0^{p(|x|)}, i) = 0^{p_i}$ if $i \le p(|x|)$, and it is 0^2 otherwise.

We define the following language in ModL

$$L' = \{\langle x, 0^{p(|x|)}, i, k\rangle \mid f(x) \equiv k(\text{mod } g(x, 0^{p(|x|)}, i)), i \leq p^2(|x|), k \leq p^2(|x|)\}.$$

In order to show that $L \in \mathrm{L}^{\mathrm{ModL}}$ we need to simulate the $\mathrm{L}^{\mathrm{GapL}}$ Turing machine for L with a $\mathrm{L}^{\mathrm{ModL}}$ computation. Clearly, it suffices to show that each GapL query $f(x)$ made by the base logarithmic space bounded deterministic Turing machine can be simulated in $\mathrm{L}^{\mathrm{ModL}}$. For each $1 \leq i \leq p(n)$, we can query L' for $\langle x, 0^{p(|x|)}, i, k\rangle$ for different values of $k \leq p^2(|x|)$ to find $f(x)(\text{mod } p_i)$.

Now, by Chinese remaindering $f(x)$ is uniquely determined by $f(x)(\text{mod } p_i)$, for $1 \leq i \leq p(n)$. Moreover, given these residues $f(x)(\text{mod } p_i)$, for $1 \leq i \leq p(n)$, it is possible to compute $f(x)$ in logarithmic space (in fact, given the residues of $f(x)(\text{mod } p_i)$, for $1 \leq i \leq p(n)$, it is possible to compute $f(x)$ in DLOGTIME-uniform TC^0 which is a very restricted form of uniformity of the uniform TC^0 complexity class). Hence a logarithmic space bounded deterministic oracle Turing machine with access to the ModL oracle L' can recover $f(x)$ for each query x. As a result it follows easily that L is in $\mathrm{L}^{\mathrm{ModL}}$. □

2.3. Some more results on directed st-connectivity

Definition 2.13. Let Σ be the input alphabet and let Γ be the output alphabet. We define FNL to be the complexity class of all functions $f : \Sigma^* \to \Gamma^*$ such that for any given input string $x \in \Sigma^*$ there exists a NL machine M which outputs $f(x)$ at the end of all of its accepting computation paths.

Definition 2.14. Let Σ be the input alphabet and let Γ be the output alphabet. We define FUL to be the complexity class of all functions $f : \Sigma^* \to \Gamma^*$ such that for any given input string $x \in \Sigma^*$ there exists a NL machine M which has atmost one accepting computation path and M outputs $f(x)$ at the end of its unique accepting computation path.

Lemma 2.15. *Let $X = \{1, \ldots, n\}$ be a set and let $\mathcal{F} \subseteq 2^X$ such that $|\mathcal{F}| \leq p(n)$ for a polynomial $p(n)$. Let $r > (n+1)^2 p^2(n)$ be a prime number and for each $1 \leq i \leq r$ and $j \in X$ define the weight function $w_i : [n] \longrightarrow \mathbb{Z}_r$ as $w_i(j) = (i^j \text{ mod } r)$. Further for each subset $Y \subseteq X$ define*

$$w_i(Y) = \Sigma_{j \in Y} w_i(j)(\text{mod } r).$$

Then there exists a weight function w_m such that $w_m(Y) \neq w_m(Y')(\text{mod } r)$ for any two distinct $Y, Y' \in \mathcal{F}$.

Proof. For any $1 \leq m \leq r$ and $Y \in \mathcal{F}$, we can interpret $w_m(Y)$ as the value of the polynomial $q_Y(z) = \Sigma_{j \in Y} z^j$ at the point $z = m$ over the field $\mathbb{Z}_r$. For $Y \neq Y'$, notice that the polynomials $q_Y(z)$ and $q_{Y'}(z)$ are distinct and their degrees are at most n. Hence, $q_Y(z)$ and $q_{Y'}(z)$ can be equal for at most n values of z in the field $\mathbb{Z}_r$. Equivalently, if $Y \neq Y'$ then $w_i(Y) = w_i(Y')$ for at most n weight functions w_i. Since there are $\binom{|\mathcal{F}|}{2}$ pairs of distinct sets in $\mathcal{F}$, it follows that there are at most $\binom{|\mathcal{F}|}{2} \cdot n < n \cdot p^2(n)$ weight functions w_i for which $w_i(Y) = w_i(Y')$ for some pair of sets $Y, Y' \in \mathcal{F}$. Since $r > n \cdot p^2(n)$, there is a weight function as claimed by the lemma. □

Theorem 2.16. *Let (G, s, t) be an input instance of* SLDAGSTCON *and let the number of directed paths from s to t in G be at most $p(n)$ for some polynomial $p(n)$, where $G = (V, E) \in$ SLDAG and $n = |E|$. Also let $E = \{1, \ldots, n\}$ and let $r > (n+1)^2 p^2(n)$ be a prime number and for each $1 \leq i \leq r$ and $j \in E$ define the weight function $w_i : E \longrightarrow \mathbb{Z}_r$ as $w_i(j) = (i^j \text{ mod } r)$. Further for each subset $Y \subseteq E$ define*

$$w_i(Y) = \Sigma_{j \in Y} w_i(j)(\text{mod } r).$$

Now let $\mathcal{F} \subseteq 2^X$ such that if $X \in \mathcal{F}$ then edges in X form a directed path from s to t in G. It is then possible to determine a weight function w_m such that $w_m(X) \neq w_m(X')(mod\ r)$ for any two distinct $X, X' \in \mathcal{F}$ and the number of directed paths from s to t in G in FNL.

PROOF. We iteratively start with the first weight function and first replace each edge in G with weight w by a directed path of length w. Let the resulting directed graph obtained be G'. It is easy to note that the number of directed paths from s to t in G is equal to the number of directed paths from s to t in G'. Since it is shown in Theorem 1.14 that SLDAGSTCON is NL-complete under logspace many-one reductions it is possible to obtain an input instance (G'', s'', t'') of SLDAGSTCON when we are given (G', s, t) as input. Upon following the proof of Theorem 1.14 we infer that the number of directed paths from s to t in H' is equal to the number of directed paths from s'' to t'' in G''. Let us initialize a counter c to 0. We consider the simple layered directed acyclic graph structure of G'' and vertices which are copies of t in all the polynomially many layers of G''. We query the NL oracle if there exists a directed path in G'' from s'' to the copy of the vertex t in each of the layers of G''. If the oracle returns "yes" then we increment the counter c by 1. For a given weight function we can compute c in $O(\log |G|)$ space. Using Lemma 2.15 it follows that there exists a weight function w_m such that $w_m(X) \neq w_m(X')(\text{mod } r)$ for any two distinct $X, X' \in \mathcal{F}$, where $1 \leq m \leq r$. Clearly any such weight function will result in the maximum value for the counter c. As a result by storing c for successive weight functions and updating it by comparison we can find the weight function w_m also. Note that the maximum value of c is the number of directed paths from s to t in G itself. Since $|G''|$ is a polynomial in $|G|$ it follows that we can find the weight function w_m in FL^{NL} which is FNL. □

THEOREM 2.17. *Let $G = (V, E)$ be a directed graph given as input and let $s, t \in V$. Also let p be a positive integer whose unary representation can be computed by a deterministic $O(\log |G|)$ space bounded Turing machine. Then we can determine if the number of directed paths from s to t in G is at least $(p + 1)$ in* FNL. *Otherwise if the number of directed paths from s to t in G is lesser than $(p + 1)$ then the* FNL *machine outputs the number of directed paths from s to t in G.*

PROOF. Due to Theorem 1.14 we know that SLDAGSTCON is NL-complete under logspace many-one reductions. We therefore follow Theorem 1.14 and obtain $H \in$ SLDAG and vertices $s', t' \in V(H)$ from G. Upon following the proof of Theorem 1.14 we infer that the number of directed paths from s to t in G is equal to the number of directed paths from s' to t' in H. Let us consider the subgraph of H induced by vertices that are in at least one directed path from s' to t' in H. Let this subgraph be H'. We use induction on the number of layers $\lambda \geq 2$ in H' and consider subgraphs of H' formed by vertices in the first λ layers of H' such that the number of directed paths from s' to vertices in layer λ of H' is at most $(p + 1)$ or a polynomial in $|G|$. Note that it is easy to compute this upperbound on the number of directed paths using $O(\log |G|)$ space. We now once again use Theorem 1.14 and Theorem 2.16 to compute the number of directed paths from s' to all the vertices in layer λ in H'. If the number of directed paths is greater than or equal to $(p + 1)$ then we move to the accepting configuration, output $(p+1)$ and stop. Otherwise finally we would have computed the number of directed paths from s' to t' in H which is lesser than $(p+1)$. We then move to the accepting configuration, output this value and stop. In all the stages of this proof throughout, we use a deterministic $O(\log |G|)$ space bounded Turing machine with access to the FNL oracle. The deterministic $O(\log |G|)$ space bounded Turing machine submits queries deterministically to the FNL oracle. In the intermediary stages, the reductions are done by submitting queries deterministically to the FNL oracle and after reading the

reply given by the oracle on the oracle tape. As a result given the input G it is possible to determine if the number of directed paths from s to t is at least $(p+1)$ is in $\mathrm{FL}^{\mathrm{FNL}}$ which is FNL. □

COROLLARY 2.18. *Verifying if there are polynomially many accepting computation paths for a* NL *machine on a given input is in* FNL.

PROOF. Let M be a NL machine and let g be the canonical logspace many-one reduction from $L(M)$ to SLDAGSTCON obtained by using the seminal result that the st-connectivity problem for directed graphs is complete for NL under logspace many-one reductions and Theorem 1.14. We note that if x is the input string then $g(x) = (G, s, t)$ and the number of accepting computation paths of M on x is equal to the number of directed paths from s to t in G. We now use Theorem 2.17 to complete the proof. □

COROLLARY 2.19. *Assume that* NL = UL. *Let $G = (V, E)$ be a directed graph given as input and let $s, t \in V$. Also let p be a positive integer whose unary representation can be computed by a deterministic $O(\log |G|)$ space bounded Turing machine. Then we can determine if the number of directed paths from s to t in G is at least $(p+1)$ in* FUL. *Otherwise if the number of directed paths from s to t in G is lesser than $(p+1)$ then the* FUL *machine outputs the number of directed paths from s to t in G.*

NOTE 2. *The number of accepting computation paths of a* NL *machine is not altered if it simulates a* NL *machine of a language $L \in$* UL *during intermediate stages to verify if some input string is in L.*

2.4. A combinatorial property of ♯L

In Theorem 2.20 upon assuming NL = UL, we show that if $f \in \sharp\mathrm{L}$ and $g \in \mathrm{FL}$ such that $g(x)$ is the unary representation of a positive integer k, where $x \in \Sigma^*$ is the input, then the number of ways of choosing exactly $k = |g(x)|$ distinct paths from amongst the $f(x)$ accepting computation paths of the NL machine corresponding to f is in $\sharp\mathrm{L}$.

THEOREM 2.20. *Assume that* NL = UL. *Let Σ be the input alphabet, $f \in \sharp\mathrm{L}$ and $g \in \mathrm{FL}$ such that, for the given input string $x \in \Sigma^*$, $g(x)$ is a positive integer k in the unary representation. Then the function $\binom{f(x)}{k} \in \sharp\mathrm{L}$.*

PROOF. Let $x \in \Sigma^*$ be the input string. Given x, we obtain the number $k = |g(x)|$ in FL. It is easy to note that k is upper bounded by a polynomial in $|x|$. It follows from our assumption that NL = UL and Definition 1.53 that there exists a NL machine M' to which if we give a directed graph G' along with two vertices s' and t' in G' as input then M' outputs "yes" at the end of the unique accepting computation path and "no" at the end of all the other computation paths if there exists a directed path from s' to t' in G'. Otherwise if there does not exist any directed path from s' to t' in G' then M' outputs "no" at the end of all of its computation paths. Now given the input x, we obtain an instance of SLDAGSTCON, say (G, s, t), using a logspace many-one reduction from Theorem 1.14. It is easy to note that the graph G that we obtain is of size polynomial in $|x|$. Once again we assume that s is in row 1 and t is in row n. Also any two paths are distinct if there exists a vertex in one of the paths that is not in the other path. It follows from Corollary 1.16 that $f(x)$ is equal to the number the directed paths from s to t in G. We associate the label (i, j) to a vertex of G if it is the j^{th} vertex in the i^{th} row of G, where $1 \leq i, j \leq n$. In the following algorithm the row number is denoted by λ and a vertex in row λ is denoted by (λ, ω), where $1 \leq \lambda, \omega \leq n$. The number of distinct paths we have chosen till row λ is denoted by φ and η denotes the

number of vertices in row λ that are in the φ distinct paths. We use nondeterminism to compute φ and η. Let us consider the SHARPLCFL algorithm described below which is based on the algorithm described in the well known Immerman-Szelepscényi Theorem and uses the method of ***double inductive counting***. Input to the SHARPLCFL algorithm is $(0^k, (G, s, t))$ where (G, s, t) is an input instance of SLDAGSTCON. If using SHARPLCFL we are able to nondeterministically choose exactly $k = |g(x)|$ distinct directed paths from s to t in G then the computation path ends in the accepting configuration. Otherwise the computation path ends in the rejecting configuration.

We note the following points about the SHARPLCFL algorithm.

Always $1 \leq \lambda \leq n$ and $1 \leq \omega \leq n$. In the SHARPLCFL algorithm, if we choose a vertex nondeterministically we verify if it is in a directed path from s to t in lines 7 and 17.

The variable η' is the number of vertices we are choosing nondeterministically in row λ and it must be equal to η after we have made nondeterministic choices on all the vertices in row λ failing which we reject the input in lines 39-42. φ' is used to verify if the number of distinct directed paths from s to t that pass through η' vertices chosen nondeterministically in row λ is atleast φ failing which we reject the input in lines 39-40.

The variable σ computed in line 16 inside the while loop from line 13 to 20 is the number of vertices chosen nondeterministically as the neighbours of (λ, ω) in row $\lambda + 1$ such that these vertices are in at least one directed path from s to t in G. Here $1 \leq \sigma \leq n$.

We compute ψ in lines 24 and 26 in FUL by Corollary 2.19 without altering the number of accepting computation paths. Note that $\psi \geq 1$ since (λ, ω) is in at least one directed path from s to t in G. After lines 24 and 26, ψ is either $\max(1, \varphi - \varphi')$ or #(directed paths from s to (λ, ω)). Therefore $1 \leq \psi \leq \varphi$ always. From lines 1, 32-33 and 43 it follows that $1 \leq \varphi \leq k$ always and so in line 30 we always have $\varphi'' \leq kn + \varphi'' \leq 2kn^2$. $1 \leq \alpha \leq \psi$ and so $0 \leq \varphi' \leq 2kn$ always. Also $0 \leq \varphi'' \leq k$ in line 4 at the begining of the while loop. Therefore $0 \leq \varphi'' \leq 2kn^2$ always.

Always $0 \leq \beta \leq \sigma$, $0 \leq \eta' \leq n$, $0 \leq \eta'' \leq n^2$ and $1 \leq \eta \leq n^2$. If $\eta > n$ then we reject the input due to the condition in lines 39 and 41. Variables φ', η' and η'' are updated in the while loop from line 4 to 38 and these values do not decrease inside this loop.

Using nondeterminism to increment η'' in lines 29 and 31 by β is to nondeterministically avoid the possibility of counting vertices in row $\lambda + 1$ that are common neighbours of two distinct vertices in row λ more than once.

In lines 27 and 29, assume that we are always nondeterministically choosing the correct value of α and β respectively in the algorithm. Then our algorithm proceeds correctly and ends in an accepting configuration if and only if we have nondeterministically chosen exactly k distinct directed paths from s to t in G. On the contrary if φ'' is updated with an incorrect value of α or if η'' is incremented by an incorrect value of β, then in those iterations of the while loop from line 4 to 38, the algorithm proceeds by assuming that an alternate set of nondeterministic choices have been made on vertices in row $\lambda + 1$ which agree with η'' and φ''.

Any two directed paths formed by nondeterministically choosing two different vertices in row λ in lines 5-6 are distinct irrespective of their neighbours nondeterministically chosen in the row $\lambda + 1$ in lines 14-15. As a result if the number of directed paths from s to t in G is lesser than k then the value of φ'' computed in line 30 is always lesser than k and these inputs are rejected in lines 41-42.

At the end of the while loop in line 38, φ'' is the number of directed paths from s to t computed nondeterministically, that we need for subsequent stages of our algorithm. Also η'' is the number of distinct

Algorithm 6 SHARPLCFL$(0^k, (G, s, t))$

1: $\lambda \leftarrow 1,\ \varphi \leftarrow 1,\ \eta \leftarrow 1$
2: **while** $\lambda \leq (n-1)$ **do**
3: $\quad \varphi' \leftarrow 0,\ \eta' \leftarrow 0,\ \varphi'' \leftarrow 0,\ \eta'' \leftarrow 0,\ \omega \leftarrow 1$
4: $\quad$ **while** $(\omega \leq n)$ **do**
5: $\quad\quad$ Nondeterministically either choose (λ, ω) or skip (λ, ω)
6: $\quad\quad$ **if** (λ, ω) is chosen nondeterministically **then**
7: $\quad\quad\quad$ **if** $(M'(G, s, (\lambda, \omega))$ returns"yes") and $(M'(G, (\lambda, \omega), t)$ returns "yes") **then**
8: $\quad\quad\quad\quad \eta' \leftarrow \eta' + 1$
9: $\quad\quad\quad\quad$ **if** $\eta' > \eta$ **then**
10: $\quad\quad\quad\quad\quad$ reject the input
11: $\quad\quad\quad\quad$ **else**
12: $\quad\quad\quad\quad\quad \sigma \leftarrow 0,\ \psi \leftarrow 0$
13: $\quad\quad\quad\quad\quad$ **while** $\exists$ a neighbour $(\lambda+1, \rho)$ of (λ, ω) that is yet to be visited **do**
14: $\quad\quad\quad\quad\quad\quad$ Nondeterministically either choose $(\lambda+1, \rho)$ or skip $(\lambda+1, \rho)$
15: $\quad\quad\quad\quad\quad\quad$ **if** $(\lambda+1, \rho)$ is chosen nondeterministically **then**
16: $\quad\quad\quad\quad\quad\quad\quad \sigma \leftarrow \sigma + 1$
17: $\quad\quad\quad\quad\quad\quad\quad$ **if** $M'(G, (\lambda+1, \rho), t)$ returns "no" **then**
18: $\quad\quad\quad\quad\quad\quad\quad\quad$ reject the input
19: $\quad\quad\quad\quad\quad\quad\quad$ **else if** $(\varphi = k$ *or* $\varphi'' = k)$ *and* $\sigma \geq 2$ **then**
20: $\quad\quad\quad\quad\quad\quad\quad\quad$ reject the input
21: $\quad\quad\quad\quad\quad$ **if** $(\sigma = 0$ *or* $\sigma > k)$ **then**
22: $\quad\quad\quad\quad\quad\quad$ reject the input
23: $\quad\quad\quad\quad\quad$ **if** $\#(\textit{directed paths from } s \textit{ to } (\lambda, \omega)) > \varphi - \varphi'$ **then**
24: $\quad\quad\quad\quad\quad\quad \psi \leftarrow \max(1, \varphi - \varphi')$
25: $\quad\quad\quad\quad\quad$ **else**
26: $\quad\quad\quad\quad\quad\quad \psi \leftarrow \#(\textit{directed paths from } s \textit{ to } (\lambda, \omega))$
27: $\quad\quad\quad\quad\quad$ Nondeterministically choose a number α from 1 *to* ψ
28: $\quad\quad\quad\quad\quad \varphi' \leftarrow \varphi' + \alpha$
29: $\quad\quad\quad\quad\quad$ Nondeterministically choose a number β from 0 to σ
30: $\quad\quad\quad\quad\quad \varphi'' \leftarrow \alpha\beta + \varphi''$
31: $\quad\quad\quad\quad\quad \eta'' \leftarrow \eta'' + \beta$
32: $\quad\quad\quad\quad\quad$ **if** $(\eta'' = 0)$ or $((\varphi = k$ or $\varphi'' > k)$ and $\eta'' > \eta)$ **then**
33: $\quad\quad\quad\quad\quad\quad$ reject the input
34: $\quad\quad\quad\quad\quad$ **if** $\varphi'' > k$ **then**
35: $\quad\quad\quad\quad\quad\quad \varphi'' \leftarrow k$
36: $\quad\quad\quad$ **else**
37: $\quad\quad\quad\quad$ reject the input
38: $\quad\quad \omega \leftarrow \omega + 1$
39: $\quad$ **if** $\lambda + 1 < n$ and $(\eta' \neq \eta$ or $\varphi' < \varphi)$ **then**
40: $\quad\quad$ reject the input
41: $\quad$ **else if** $\lambda + 1 = n$ and $(\eta' \neq \eta$ or $\varphi'' \neq k)$ **then**
42: $\quad\quad$ reject the input
43: $\quad \lambda \leftarrow \lambda + 1,\ \varphi \leftarrow \varphi'',\ \eta \leftarrow \eta''$
44: accept the input

vertices that are in row $\lambda + 1$ in φ'' distinct directed paths from s to t in G which is also computed nondeterministically.

Now assume that the number of directed paths from s to t in G is at least k. The cases where we reject the input since the nondeterministic choices made on the neighbours of (λ, ω) in row $\lambda + 1$ results in increasing the number of distinct directed paths nondeterministically chosen to be greater than k is in lines 19, 20, 32 and 33.

Lines 19 and 20 are pertaining to the case when we are in vertex (λ, ω) in row λ and we are visiting the neighbours of (λ, ω) in row $\lambda+1$. In this case we have already nondeterministically chosen k distinct directed paths from s to t and we have also chosen vertices in row $\lambda+1$ in excess that results in increasing the number of distinct directed paths chosen nondeterministically from s to t to be greater than k. Similarly in lines 32 and 33 we have the case when we have chosen a (λ, ω) in row λ and we have visited all the neighbours of (λ, ω) in row $\lambda + 1$. If ($\varphi = k$ or $\varphi'' > k$) and $\eta'' > \eta$ then it implies that we have chosen more than k distinct directed paths from s to t in G that most recently includes directed paths that pass through (λ, ω) and η'' distinct neighbours of (λ, ω) in row $\lambda + 1$ and so in lines 32 and 33 we reject the input.

The case when we reject the input since we have not chosen η vertices in row $\lambda < n - 1$ or at least φ distinct directed paths till row $\lambda < n - 1$ is in lines 39 and 40. The case when we reject the input since we have not chosen exactly k distinct directed paths from s to t but we have moved till row $n - 1$ is in lines 41 and 42.

In the SHARPLCFL algorithm, since we keep track of only a constant number of variables all of which take non-negative integer values and the values of these variables are upper bounded by a polynomial in the size of the graph G, which is once again polynomial in the input size $|x|$ we get a NL machine that executes the SHARPLCFL algorithm. Since we have assumed that NL = UL and we use M' to verify if there exists a directed path from a vertex s' to a vertex t' in lines 13 and 23, it follows that the number of accepting computation paths of the NL machine described by our SHARPLCFL algorithm does not get altered upon simulating M' (also see Note 2). As a result it follows that the NL machine described by the SHARPLCFL algorithm stops in an accepting state if and only if we start from vertex s in row 1 and reach vertex t in row n via exactly k distinct directed paths. Now as mentioned in Corollary 1.16, since $f(x)$ is equal to the number of directed paths from s to t in G and since we have assumed NL = UL, it follows that the number of accepting computation paths of this NL machine is $\binom{f(x)}{k}$. When $f(x) < k$ this NL machine has no accepting computation paths and so we get $\binom{f(x)}{k} = 0$. This shows that $\binom{f(x)}{k} \in \sharp\text{L}$. □

2.5. Characterization of ModL

In this section we study the complexity class ModL and obtain a characterization of ModL that shows that a $\sharp$L function f and a FL function g that outputs a prime number in unary representation are sufficient to decide if a given input x is in a language $L \in$ ModL, however under the assumption that NL = UL. More precisely we are able to show a characterization of ModL in Theorem 2.22 that if we assume NL = UL and we have a language $L \subseteq \Sigma^*$ with $L \in$ ModL then we can decide whether an input $x \in \Sigma^*$ is in L using $f \in \sharp$L and $g \in$ FL where $g(x)$ is a prime number p that is output by g in the unary representation such that if $x \in L$ then $f(x) \equiv 1 (\text{mod } p)$ and if $x \notin L$ then $f(x) \equiv 0 (\text{mod } p)$. As an immediate consequence of Theorem 2.22, we show that ModL is closed under complement assuming NL = UL in Corollary 2.23.

LEMMA 2.21. *Let $L \in$ ModL. Then there exists a function $f \in \sharp$L and a function $g \in$ FL such that on any input string x,*

- *$g(x) = 0^{p^e}$ for some prime p and a positive integer e, and*
- *$x \in L$ if and only if $f(x) \not\equiv 0 (mod\ p^e)$.*

PROOF. Let $L \in$ ModL be witnessed by functions $f' \in$ GapL and $g \in$ FL as in Definition 2.11. Now given $f' \in$ GapL there exists $f_1, f_2 \in \sharp$L such that on any input string x we have $f'(x) = f_1(x) - f_2(x)$. Consider $f(x) = f_1(x) + (p^e - 1) f_2(x)$. Since $\sharp$L is closed under multiplication by a FL function that outputs a positive

integer and also under addition, we have $f(x) \in \sharp\text{L}$. Moreover on a given input x, we have $f'(x) \not\equiv 0 (\text{mod } p^e)$ if and only if $f'(x) = f_1(x) - f_2(x) \not\equiv 0 (\text{mod } p^e)$ if and only if $f(x) = f_1(x) + (p^e - 1)f_2(x) \not\equiv 0 (\text{mod } p^e)$. As a result we can replace the GapL function f' by the $\sharp$L function f to decide if any given input string x is in L. □

THEOREM 2.22. *Assume that* NL = UL *and let* $L \in$ ModL. *Then there exists a function* $f \in \sharp\text{L}$ *and a function* $g \in$ FL *such that on any input string* x,

- $g(x) = 0^p$ *for some prime* $p > 0$, *and,*
- *if* $x \in L$ *then* $f(x) \equiv 1 (mod\ p)$,
- *if* $x \notin L$ *then* $f(x) \equiv 0 (mod\ p)$.

PROOF. Let $L \in$ ModL. It follows from Lemma 2.21 that there exists $f' \in \sharp\text{L}$ and $g' \in$ FL such that on any input string x we have $g'(x) = 0^{p^e}$ for some prime p and a positive integer e, and $x \in L$ if and only if $f'(x) \not\equiv 0 (\text{mod } p^e)$. Let g be a FL function that outputs the prime p in unary when given the input x. Now assume that there exists a $f'' \in \sharp\text{L}$ such that $f'(x) \not\equiv 0 (\text{mod } p^e)$ if and only if $f''(x) \not\equiv 0 (\text{mod } p)$. Then $x \in L$ if and only if $f''(x) \not\equiv 0 (\text{mod } p)$. Define $f(x) = (f''(x))^{(p-1)}$. Using Fermat's Little Theorem we have, if $x \in L$ then $f(x) \equiv 1 (\text{mod } p)$. Otherwise if $x \notin L$ then $f(x) \equiv 0 (\text{mod } p)$. We therefore prove the theorem statement if we define the function f'' such that $f'(x) \not\equiv 0 (\text{mod } p^e)$ if and only if $f''(x) \not\equiv 0 (\text{mod } p)$.

It is easy to see that we can compute the largest power of p that divides $p^e = |g'(x)|$ in FL. If $e = 1$ then we define $f'' = f'$ where f' is the $\sharp$L function in Lemma 2.21. It is clear that on an input string x we have $g(x) = g'(x) = 0^p$ for some prime p and $x \in L$ if and only if $f'(x) \not\equiv 0 (\text{mod } p^e)$ which is true if and only if $f''(x) \not\equiv 0 (\text{mod } p)$. Otherwise $e \geq 2$. Here we have $g'(x) = 0^{p^e}$ and $g(x) = 0^p$. Now we use induction on $(e-1)$ to define f''. Inductively assume that for $1 \leq i \leq (e-1)$ we have functions $f_i \in \sharp\text{L}$ such that $f'(x) \not\equiv 0 (\text{mod } p^i)$ if and only if $f_i(x) \not\equiv 0 (\text{mod } p)$. For the case when $(e-1) = 1$ we can have $f_{e-1} = f'$. Then it is clear that given an input string x, we have $f'(x)$ is divisible by p^e if and only if

(1) $f'(x)$ is divisible by p^{e-1}, and
(2) the coefficient of p^{e-1} in the base-p expansion of $f'(x)$ is zero.

Here Condition 2 stated above is equivalent to $\binom{f'(x)}{p^{e-1}} \equiv 0 (\text{mod } p)$. Therefore $f'(x) \not\equiv 0 (\text{mod } p^e)$ if and only if $\{f'(x) \not\equiv 0 (\text{mod } p^{e-1})$ or $\binom{f'(x)}{p^{e-1}} \not\equiv 0 (\text{mod } p)\}$ which is true if and only if $\{f_{e-1}(x) \not\equiv 0 (\text{mod } p)$ or $\binom{f'(x)}{p^{e-1}} \not\equiv 0 (\text{mod } p)\}$. Let $f'_e(x) = (f_{e-1}(x))^{p-1}\binom{f'(x)}{p^{e-1}}^{p-1} + ((f_{e-1}(x))^{p-1} + p - 1)\binom{f'(x)}{p^{e-1}}^{p-1} + (f_{e-1}(x))^{p-1}(\binom{f'(x)}{p^{e-1}}^{p-1} + (p-1))$. Now define $f_e(x) = (f'_e(x))^{p-1}$. Using Theorem 2.20 and closure properties of $\sharp$L it follows that $f_e \in \sharp\text{L}$. Moreover on an input x, if $f'(x) \equiv 0 (\text{mod } p^e)$ then $f_e(x) \equiv 0 (\text{mod } p)$. On the other hand if $f'(x) \not\equiv 0 (\text{mod } p^e)$ then we consider the following cases. Case 1: $f'(x) \equiv 0 (\text{mod } p^{e-1})$: Then $f_{e-1}(x) \equiv 0 (\text{mod } p)$ and we also have $\binom{f'(x)}{p^{e-1}} \not\equiv 0 (\text{mod } p)$. As a result from the definition of $f_e(x)$ we get $f_e(x) \equiv 1 (\text{mod } p)$.
Case 2: $f'(x) \not\equiv 0 (\text{mod } p^{e-1})$: Then $f_{e-1}(x) \not\equiv 0 (\text{mod } p)$ and we can have either $\binom{f'(x)}{p^{e-1}} \not\equiv 0 (\text{mod } p)$ or $\binom{f'(x)}{p^{e-1}} \equiv 0 (\text{mod } p)$. But in both of these cases we get $f_e(x) \equiv 1 (\text{mod } p)$. As a result defining $f'' = f_e$ we complete the proof. □

COROLLARY 2.23. *Assume that* NL = UL. *Then* ModL *is closed under complement.*

PROOF. Let $L \in$ ModL. Then by Theorem 2.22 there exists $f \in \sharp\text{L}$ and $g \in$ FL such that on any input x, we have $g(x) = 0^p$ for some prime p and if $x \in L$ then $f(x) \equiv 1 (\text{mod } p)$. Otherwise if $x \notin L$ then $f(x) \equiv 0 (\text{mod } p)$.

Let $h(x) = (f(x)+(p-1))^{(p-1)}$. Using closure properties of $\sharp$L it follows that $h(x) \in \sharp$L. Using Fermat's Little Theorem we have, if $x \in L$ then $h(x) \equiv 0 (\text{mod } p)$ and if $x \notin L$ then $h(x) \equiv 1 (\text{mod } p)$. Clearly this shows $\overline{L} \in$ ModL or that ModL is closed under complement. □

2.6. Closure properties of ModL

As a consequence of the characterization of ModL that we have shown in Theorem 2.22, it is shown in Corollary 2.23 that ModL is closed under complement under the assumption that NL = UL. We continue to study the complexity class ModL and obtain its closure properties which follow due to the above stated characterization of ModL.

2.6.1. An unconditional closure property of ModL.

DEFINITION 2.24. Let Σ be the input alphabet such that $\{0,1\} \subseteq \Sigma$ and let $L_1, L_2 \subseteq \Sigma^*$. We define the join of L_1 and L_2 to be $L_1 \vee L_2 = \{x1 | x \in L_1\} \cup \{y0 | y \in L_2\}$.

THEOREM 2.25. *If* $\{0,1\} \subseteq \Sigma$ *and* $L_1 \in$ ModL *then* $L_1 \vee L_2 \in$ ModL.

PROOF. Let $L_i \in$ ModL and assume that we decide if an input string $x \in \Sigma^*$ is in L_i using functions $f_i \in$ GapL and $g_i \in$ FL where $1 \leq i \leq 2$ respectively. We assume without loss of generality that the size of any input string x is at least 1. Let $x = x_1 x_2 \cdots x_{n+1}$ and let $y = x_1 \cdots x_n$ where $n \in \mathbb{Z}^+$. Also let

$$f(x) = \begin{cases} f_1(y) & \text{if } x_{n+1} = 1 \\ f_2(y) & \text{otherwise if } x_{n+1} = 0, \end{cases}$$

and

$$g(x) = \begin{cases} g_1(y) & \text{if } x_{n+1} = 1 \\ g_2(y) & \text{otherwise if } x_{n+1} = 0. \end{cases}$$

It is easy to note that $f \in$ GapL and $g \in$ FL. Since $|g_1(x)|$ or $|g_2(x)|$ is always a prime power on any input string $x \in \Sigma^*$ it follows that $|g(x)|$ is also a prime power. Now $f(x) \not\equiv 0 (\text{mod } |g(x)|)$ for any input $x = yx_{n+1}$ if and only if exactly one of the following is true: ($x_{n+1} = 1$ and $y \in L_1$) or ($x_{n+1} = 0$ and $y \in L_2$). Clearly this shows that $L_1 \vee L_2 \in$ ModL and that we can decide if any input string $x \in \Sigma^*$ is in $L_1 \vee L_2$ using $f \in$ GapL and $g \in$ FL which completes the proof. □

In Theorem 2.37(1) we show that ModL is closed under $\leq_m^{\text{L}}$ reductions and similarly in Theorem 2.37(2) we show that ModL is closed under $\leq_m^{\text{UL}}$ reductions. Using Theorem 2.22 and Corollary 2.23 we show in Theorem 2.38(1) that ModL is closed under $\leq_{1-\text{tt}}^{\text{L}}$ reductions assuming NL = UL. We also show that if NL = UL then ModL is closed under $\leq_{1-\text{tt}}^{\text{UL}}$ reductions in Theorem 2.38(2). The following table lists the closure properties of ModL that are known.

Closure property	*Assumptions*	*Reference*
$\vee$ (join)	unconditional	Theorem 2.25
complement	NL = UL	Corollary 2.23
$\leq_m^{\text{L}}$	unconditional	Theorem 2.37(1)
$\leq_m^{\text{UL}}$	unconditional	Theorem 2.37(2)
$\leq_{1-\text{tt}}^{\text{L}}$	NL = UL	Theorem 2.38(1)
$\leq_{1-\text{tt}}^{\text{UL}}$	NL = UL	Theorem 2.38(2)

Table 1: Closure properties of ModL

We also show that $\text{ModL}^{\text{UL}} = \text{ModL}$ assuming UL = co-UL in Theorem 2.39. As a consequence of these results we show in Corollary 2.40 that $\text{UL}^{\text{ModL}}_{1-\text{tt}} = \text{ModL}^{\text{UL}} = \text{ModL}$ assuming NL = UL.

2.7. Reducibilities of ModL

We recall the definitions stated in Section 1.3 of Chapter 1.

2.7.1. Reducibilities.

DEFINITION 2.26. Let Σ be the input alphabet and let $L_1, L_2 \subseteq \Sigma^*$. We say that L_1 is logspace conjunctive truth-table reducible to L_2, denoted by $L_1 \leq^{\text{L}}_{\text{ctt}} L_2$, if there exists $f \in \text{FL}$ such that given an input $x \in \Sigma^*$ of length n we have $f \in \text{FL}$ such that $f(x) = \{y_1 \ldots, y_{p(n)}\}$ and $x \in L_1$ if and only if $y_i \in L_2$ for every $1 \leq i \leq p(n)$ where $p(n)$ is a polynomial in n.

DEFINITION 2.27. Let Σ be the input alphabet and let $L_1, L_2 \subseteq \Sigma^*$. We say that L_1 is logspace k-conjunctive truth-table reducible to L_2, denoted by $L_1 \leq^{\text{L}}_{k-\text{ctt}} L_2$, if there exists $f \in \text{FL}$ such that given an input $x \in \Sigma^*$ of length n $f(x) = \{y_1 \ldots, y_k\}$ and $x \in L_1$ if and only if $y_i \in L_2$ for every $1 \leq i \leq k$.

DEFINITION 2.28. Let Σ be the input alphabet and let $L_1, L_2 \subseteq \Sigma^*$. We say that L_1 is logspace disjunctive truth-table reducible to L_2, denoted by $L_1 \leq^{\text{L}}_{\text{dtt}} L_2$, if there exists $f \in \text{FL}$ such that given an input $x \in \Sigma^*$ of length n we have $f(x) = \{y_1, \ldots, y_{p(n)}\}$ and $x \in L_1$ if and only if $y_i \in L_2$ for at least one $1 \leq i \leq p(n)$ where $p(n)$ is a polynomial in n.

DEFINITION 2.29. Let Σ be the input alphabet and let $L_1, L_2 \subseteq \Sigma^*$. We say that L_1 is logspace k-disjunctive truth-table reducible to L_2 for some $k \in \mathbb{Z}^+$, $k \geq 1$, denoted by $L_1 \leq^{\text{L}}_{k-\text{dtt}} L_2$, if there exists $f \in \text{FL}$ such that given an input $x \in \Sigma^*$ of length n we have $f(x) = \{y_1, \ldots, y_k\}$ and $x \in L_1$ if and only if $y_i \in L_2$ for at least one $1 \leq i \leq k$.

DEFINITION 2.30. Let Σ be the input alphabet and let $L_1, L_2 \subseteq \Sigma^*$. We say that L_1 is logspace 1-truth-table reducible to L_2 and denoted by $L_1 \leq^{\text{L}}_{1-\text{tt}} L_2$ if there exists a $O(\log n)$ space bounded Turing machine M^{L_2} that has access to L_2 as an oracle such that M^{L_2} decides if any given input $x \in \Sigma^*$ is in L_1 by making exactly one query to the oracle L_2, where $n = |x|$.

DEFINITION 2.31. Let Σ be the input alphabet and let $L_1, L_2 \subseteq \Sigma^*$. We say that $L_1 \leq^{\text{UL}}_{m} L_2$ if there exists a FNL Turing machine M such that on any input $x \in \Sigma^*$ the number of accepting computation paths of $M(x)$ is at most 1. Also if $x \in L_1$ then there exists an accepting computation path of M on input x such that if we obtain $y \in \Sigma^*$ as the output along the accepting computation path then $y \in L_2$. If $x \notin L_1$ then either M does not have any accepting computation path on input x or if there exists an accepting computation path of M on input x and $y \in \Sigma^*$ is the output along this computation path then $y \notin L_2$.

DEFINITION 2.32. Let Σ be the input alphabet and let $L_1, L_2 \subseteq \Sigma^*$. We say that $L_1 \leq^{\text{UL}}_{1-\text{tt}} L_2$ if there exists a NL machine M^{L_2} with access to the oracle L_2 such that on any input $x \in \Sigma^*$ the number of accepting computation paths of $M^{L_2}(x)$ is at most 1. Also M^{L_2} submits exactly one query $q_x \in \Sigma^*$ to the L_2 oracle on any input $x \in \Sigma^*$ to decide if $x \in L_1$. We assume that the query is submitted by the NL machine M^{L_2} in a deterministic manner to the L_2 oracle according to the Ruzzo-Simon-Tompa oracle access mechanism stated above. Here if $x \in L_1$ then there exists exactly one accepting computation path of M^{L_2} on input x. Otherwise if $x \notin L_1$ then M^{L_2} rejects the input x on all of its computation paths.

2.7.2. ModL. We recall the definiton of ModL stated in Definition 2.11.

DEFINITION 2.33. Let Σ be the input alphabet and let $A \in \Sigma^*$. We define the complexity class ModL^A to be the class of all languages $L \in \Sigma^*$ for which there exists a NL machine M^A with access to the oracle A and a function $g \in \text{FL}$ such that on any input $x \in \Sigma^*$ if $f(x) = gap_{M^A}(x)$, we have

- $g(x) = 1^{p^e}$ for some prime p and a positive integer e, and
- $x \in L \Leftrightarrow f(x) \not\equiv 0 (\text{mod } p^e)$.

Here the NL machine M^A submits queries to the oracle A in a deterministic manner according to the Ruzzo-Simon-Tompa oracle access mechanism stated above. Along any computation path the NL machine M^A ends in an accepting state or in a rejecting state depending on the input x and the replies given by the oracle A.

DEFINITION 2.34. Let Σ be the input alphabet and let $L_1, L_2 \in \Sigma^*$. We define the complexity class $\text{UL}_{1-\text{tt}}^{\text{ModL}} = \{L_1 | L_1 \leq_{1-\text{tt}}^{\text{UL}} L_2, \text{ where } L_2 \in \text{ModL}\}$.

DEFINITION 2.35. Let Σ be the input alphabet and let $L_1, L_2 \in \Sigma^*$. We define the complexity class $\text{ModL}^{\text{UL}} = \{L_1 | L_1 \in \text{ModL}^{L_2}, \text{ where } L_2 \in \text{ModL}\}$.

LEMMA 2.36. *Let Σ be the input alphabet and let $g \in \text{FL}$ such that on any input $x \in \Sigma^*$, we have $g(x)$ to be a prime number p in the unary notation that depends on the input x. Then there exists a* NL *machine M such that the lexicographically least $|g(x)|$ computation paths are exactly the only accepting computation paths in the computation tree of M.*

PROOF. Let $x \in \Sigma^*$ such that $n = |x|$ and let $g(x) = 0^p$ for some prime $p \in \mathbb{Z}^+$. Let $k \in \mathbb{Z}^+$ such that $2^{k-1} \leq p < 2^k$. It is easy to see that $k, p \in O(\log n)$. The NL machine M simulates g on input x and computes $p = |g(x)|$. The NL machine M then non-deterministically makes k choices and accepts in a computation path if the non-deterministic choices are a binary string less than or equal to p. M rejects along all other computation paths. □

Apart from the chararacterization of ModL shown in Theorem 2.22, we also need Lemma 2.21.

THEOREM 2.37. *Let Σ be the input alphabet and let $L_1, L_2 \subseteq \Sigma^*$. Also let $L_2 \in$ ModL. Now*

(1) *if $L_1 \leq_m^{\text{L}} L_2$ then $L_1 \in$ ModL, and*
(2) *if $L_1 \leq_m^{\text{UL}} L_2$ then $L_1 \in$ ModL.*

PROOF. (1) Let $L_2 \in \text{ModL}$ and let $f_2 \in \text{GapL}$ and $g_2 \in \text{FL}$ be functions using which we decide if an input $x \in \Sigma^*$ is in L_2. Also let $L_1 \leq_m^{\leq^{\text{L}}_m} L_2$ using $f \in \text{FL}$. If $x \in \Sigma^*$ is the input then $x \in L_1$ if and only if $f(x) \in L_2$. But $L_2 \in \text{ModL}$ and therefore $f(x) \in L_2$ if and only if $f_2(f(x)) \not\equiv 0 (\text{mod } p^e)$ where $p^e = |g_2(f(x))| \in \mathbb{Z}^+$ where $p, e \in \mathbb{Z}^+$ with p being a prime and $e \geq 1$. Now it is easy to note that $(f_2 \circ f) \in \text{GapL}$ and $(g_2 \circ f) \in \text{FL}$. Clearly $g_2(f(x))$ is a prime power in the unary representation for any $x \in \Sigma^*$. As a result if $x \in \Sigma^*$ then $x \in L_1$ if and only if $f_2(f(x)) \not\equiv 0 (\text{mod } p^e)$ where $p^e = |g_2(f(x))|$ which shows $L_1 \in \text{ModL}$.

(2) We recall the definition $\leq_m^{\text{UL}}$ from Definition 2.31. The proof of this assertion is similar to Theorem 2.37(1). We use Lemma 2.21 for $L_2 \in \text{ModL}$ and assume that there exists $f \in \sharp\text{L}$ and $g \in \text{FL}$ such that on any input $x \in \Sigma^*$ we have $g(x) = 1^{p^e}$ where $p, e \in \mathbb{Z}^+$, p is a prime, $e \geq 1$ and $x \in L_2$ if and only if $f(x) \not\equiv 0 (\text{mod } p^e)$. We know that the $\leq_m^{\text{UL}}$ reduction has at most one accepting

computation path. Therefore if the computation path we obtain is an accepting computation path and $y \in \Sigma^*$ is output along this accepting computation path, then simulating the NL machine for $L_2 \in \text{ModL}$ with y as the input shows that the congruence relation based on the $\sharp\text{L}$ function f and the FL function g that is used to decide if any input $x \in \Sigma^*$ is in L_2 can also be used for deciding if $x \in L_1$. Otherwise if the computation path of the $\leq_m^{\text{UL}}$ reduction that we obtain is a rejecting computation path then using Lemma 2.36 it follows that our NL machine can make sufficiently many non-deterministic choices so that the number of accepting computation paths that we obtain at the end of each of these rejecting computation paths is divisible by $|g(x)|$. This shows $L_1 \in \text{ModL}$.

□

THEOREM 2.38. *Let Σ be the input alphabet and let $L_1, L_2 \subseteq \Sigma^*$. Also let $L_2 \in \text{ModL}$ and assume that* NL = UL. *Now*

(1) *if $L_1 \leq_{1-\text{tt}}^{\text{L}} L_2$ then $L_1 \in \text{ModL}$, and*
(2) *if $L_1 \leq_{1-\text{tt}}^{\text{UL}} L_2$ then $L_1 \in \text{ModL}$.*

PROOF. (1) We recall the definition of $\leq_{1-\text{tt}}^{\leq_m^{\text{L}}}$ from Definition 2.30. Let $L_1 \leq_{1-\text{tt}}^{\leq_m^{\text{L}}} L_2$ using $f \in \text{FL}$ that makes exactly one query to the oracle L_2. In other words, f correctly decides if an input $x \in \Sigma^*$ is in L_1 by making exactly one query to the oracle L_2. Also let $q_x \in \Sigma^*$ be the query string that is generated by f for the input $x \in \Sigma^*$. Clearly f decides if $x \in L_1$ based on x and the reply of the oracle L_2 when the oracle L_2 is given the query q_x as input. Therefore let $f' \in \text{FL}$ be the function that generates and outputs the query q_x generated by f when it is given the input x. Also let $g' \in \text{FL}$ be such that $g'(x, \chi_{L_2}(q_x)) = f(x)$ where χ_{L_2} is the characteristic function of L_2. It is clear that g' correctly decides whether any input $x \in L_1$ if the reply of the oracle L_2 is also given to it.

Since $L_2 \in \text{ModL}$ and we have assumed NL = UL, it follows from Theorem 2.22 that there exists $f_2 \in \sharp\text{L}$ and $g_2 \in \text{FL}$ such that on any input $x \in \Sigma^*$ we have if $x \in L_2$ then $f_2(x) \equiv 1 (\text{mod } p)$ and if $x \notin L_2$ then $f_2(x) \equiv 0 (\text{mod } p)$, where $g_2(x) = 1^p$, $p \in \mathbb{Z}^+$ and p is a prime. Let us consider the function $(g_2 \circ f')$. Given an input $x \in \Sigma^*$, $g_2(f(x)) = 1^p$, for some prime $p \in \mathbb{Z}^+$ which depends on the query $f'(x)$ generated from the input x. It is clear that $(g_2 \circ f') \in \text{FL}$. Consider the function $(g' \circ f_2 \circ f')$ and let M denote the NL machine corresponding to it. The NL machine M considers the output of a computation path of the NL machine corresponding to $(f_2 \circ f')(x)$ as $\chi_{L_2}(q_x)$. Since $g' \in \text{FL}$ it follows that the output of M is the same along all the accepting computation paths of the NL machine corresponding to $(f_2 \circ f')$. In other words, either M accepts along all the accepting computation paths of the NL machine corresponding to $(f_2 \circ f')$ or M rejects along all the accepting computation paths of the NL machine corresponding to $(f_2 \circ f')$. Similarly either M accepts along all the rejecting computation paths of the NL machine corresponding to $(f_2 \circ f')$ or M rejects along all the rejecting computation paths of the NL machine corresponding to $(f_2 \circ f')$.

Let $x \in \Sigma^*$ be the input and let $g_2(f'(x)) = 1^p$. We now define a NL machine M' that makes $(p-1)$ simulations of M on the input x. In addition M' also stores if the computation path chosen is the lexicographically least in its computation tree on any input $x \in \Sigma^*$. Now if the output of M along any of its computation paths is the same as the output of the computation path of the NL machine corresponding to $(f_2 \circ f')$ on the input x then M' makes $(p-1)$ times the simulation

of M on input x and stops. In this case, for a given input $x \in \Sigma^*$, if $L_1 \leq^{\mathrm{L}}_{1-\mathrm{tt}} L_2$ then along every computation path the reduction is similar to a logspace many-one reduction from L_1 to L_2. In this case using Fermat's Little Theorem, it follows from our observations that if $x \in L_1$ then $acc_{M'}(x) \equiv 1 (\mathrm{mod}\ p)$ and if $x \notin L_1$ then $acc_{M'}(x) \equiv 0 (\mathrm{mod}\ p)$. This shows $L_1 \in \mathrm{ModL}$. On the other hand if we have the output of M along any of its computation paths is the complement of the output of the computation path of the NL machine corresponding to $(f_2 \circ f')$ on a given input $x \in \Sigma^*$ then also M' makes $(p-1)$ simulations of M on input x. However in these $(p-1)$ simulations of M by M', along any of the computation paths of M' that is not the lexicographically least computation path we assume that M' stops with the output of M on input x. Along the lexicographically least computation path that we obtain in these $(p-1)$ simulations of M by M' we assume that M' makes sufficiently many non-deterministic choices at the end of this computation path as in Lemma 2.36 so that we obtain $acc_{M'}(x) = acc_M(x)^{p-1} + (p-1)$. In this case using Fermat's Little Theorem, it follows from our observations that if $x \in L_1$ then $acc_{M'}(x) \equiv 0 (\mathrm{mod}\ p)$ and if $x \notin L_1$ then $acc_{M'}(x) \not\equiv 0 (\mathrm{mod}\ p)$ which implies $\overline{L_1} \in \mathrm{ModL}$. We now use our assumption that NL = UL and so it follows from Corollary 2.23 that $L_1 \in \mathrm{ModL}$ and this completes the proof.

(2) We recall the definition of $\leq^{\mathrm{UL}}_{1-\mathrm{tt}}$ from Definition 2.32. We follow the Ruzzo-Simon-Tompa oracle access mechanism stated in Section 1.1.4 and generate the query string $q_x \in \Sigma^*$ given the input $x \in \Sigma^*$ in a deterministic manner using $O(\log n)$ space. Our proof follows from our assumption that NL = UL and a case analysis similar to the proof of Theorem 2.38(1) depending on whether the $\leq^{\mathrm{UL}}_{1-\mathrm{tt}}$ reduction is similar to a $\leq^{\mathrm{UL}}_{m}$ reduction or if the output of the computation paths of the NL machine that is used in the reduction is the complement of the output of the computation paths of the NL machine for the oracle L_2 on the query string q_x.

□

THEOREM 2.39. *Assume that* UL = co-UL. *Then* $\mathrm{ModL}^{\mathrm{UL}} \subseteq \mathrm{ModL}$.

PROOF. The containment $\mathrm{ModL} \subseteq \mathrm{ModL}^{\mathrm{UL}}$ follows immediately from the definition of ModL. To show the converse, assume that UL = co-UL. Let Σ be the input alphabet and let $L_1, L_2 \subseteq \Sigma^*$. Also let $L_2 \in \mathrm{UL}$ and $L_1 \in \mathrm{ModL}^{L_2}$. As a result using Lemma 2.21 for L_1 it follows from Definition 2.35 that on any input $x \in \Sigma^*$ there exists a NL machine M^{L_2} with oracle L_2 such that there exists $f \in \sharp\mathrm{L}$, where $f(x) = acc_{M^{L_2}}(x)$, and $g \in \mathrm{FL}$ and we have $x \in L_1$ if and only if $f(x) \not\equiv 0 (\mathrm{mod}\ |g(x)|)$ where $g(x) = 1^{p^e}$ with $p, e \in \mathbb{Z}^+$, p being a prime and $e \geq 1$. Here we assume that the queries that are submitted to the oracle L_2 by M are generated in a deterministic manner based on the length of the input according to the Ruzzo-Simon-Tompa oracle access mechanism stated in Section 1.1.4. As a consequence we assume that M submits the queries to the L_2 oracle and also obtains the reply of the oracle on the oracle tape before it starts any other computation on the input. In addition since M is $O(\log n)$ space bounded, for any given input $x \in \Sigma^*$ such that $n = |x|$, it is clear that the size of each query is at most n^j and the number of queries that M can submit to L_2 is at most n^k where $j, k \in \mathbb{Z}^+$. Let us denote the i^{th} query that is generated by M as $q_x^{(i)} \in \Sigma^*$ where $1 \leq i \leq n^k$.

The NL machine M' that we define for L_1 simulates M on the input x. However unlike M the NL machine M' does not have access to any oracle. However M' needs to obtain the reply of the L_2 oracle for the queries that M submits in its computation path. For this purpose we assume that M' generates the i^{th} query $q_x^{(i)}$ in a deterministic manner during its simulation of M on input x, where $1 \leq i \leq n^k$. For every query $q_x^{(i)}$ that is generated by M', to determine if $q_x^{(i)} \in L_2$, the NL machine M' simulates the NL machine

M_{L_2} corresponding to $L_2 \in$ UL. Clearly if $q_x^{(i)} \in L_2$ then M_{L_2} has exactly one accepting computation path. If the simulation of M_{L_2} by M' results in this accepting computation path then M' assumes the output of the oracle query that M would have obtained when it submits the query $q_x^{(i)}$ as input to the L_2 oracle to be "yes" and it continues its simulation of M on input x. Otherwise let us assume that the simulation of M_{L_2} by M' results in a rejecting computation path. In this case to test if $q_x^{(i)} \in \overline{L_2}$ we use the assumption that UL = co-UL and hence allow M' to simulate the NL machine $M_{\overline{L_2}}$ for $\overline{L_2}$ (which is also a language in UL) on input $q_x^{(i)}$ to test if this simulation results in an accepting computation path. If M' ends in an accepting computation path in this simulation then it implies $q_x^{(i)} \in \overline{L_2}$. Therefore M' assumes that the reply of the L_2 oracle is "no" on the query $q_x^{(i)}$ and continues its simulation of M on input x. Otherwise if both these simulations of M' on input $q_x^{(i)}$ results only in rejecting computation paths then M' makes sufficiently many non-deterministic choices in the computation tree and ends in a rejecting state in all these computation paths. For any query string $q_x^{(i)}$ that is generated by M' along any of its computation paths we either have $q_x^{(i)} \in L_2$ or we have $q_x^{(i)} \in \overline{L_2}$, where $1 \leq i \leq n^k$. Now it follows from the definition of M' that we can determine if $q_x^{(i)} \in L_2$ by simulating M_{L_2} or if necessary $M_{\overline{L_2}}$ on $q_x^{(i)}$ during its computation on the input x, where $1 \leq i \leq n^k$. Due to our assumption that UL = co-UL it also follows that the number of computation paths in these simulations that correctly determine if $q_x^{(i)} \in L_2$ is exactly one where $1 \leq i \leq n^k$. Apart from simulating M_{L_2} and $M_{\overline{L_2}}$ since the computation of M' is identical to M on input x it follows that $acc_{M'}(x) = f(x)$. As a result we have $x \in L_1$ if and only if $acc_{M'}(x) \not\equiv 0 (\text{mod } |g(x)|)$. This shows $L_1 \in$ ModL due to Lemma 2.21. We therefore get $\text{ModL}^{\text{UL}} = \text{ModL}$. □

COROLLARY 2.40. *Assume that* NL = UL. *Then* $\text{UL}_{1-\text{tt}}^{\text{ModL}} = \text{ModL}^{\text{UL}} = \text{ModL}$.

PROOF. Since we know that NL is closed under complement from Theorem 1.22, our assumption that NL = UL implies UL = co-UL. Proof of our result now follows from Theorem 2.38(2) and Theorem 2.39. □

2.8. Properties of the Probabilistic Logarithmic space bounded counting class (PL)

PROPOSITION 2.41. *Let* Σ *be the input alphabet. The class* PL *consists of those languages* $L \subseteq \Sigma^*$ *such that for some* GapL *function* f *and all* $x \in \Sigma^*$

- *if* $x \in L$ *then* $f(x) > 0$
- *if* $x \notin L$ *then* $f(x) < 0$.

PROOF. For a GapL function f, the GapL function $2f(x) - 1$ has the same sign as $f(x)$ for positive and negative values of $f(x)$ and takes value -1 for $f(x) = 0$. □

COROLLARY 2.42. PL *is closed under complement.*

LEMMA 2.43. *Define*

$$P_n(x) = (x-1)\prod_{i=1}^{n}(x-2^i)^2.$$
$$A_n(x) = P_n(-x))^{\frac{n}{2}+1} - (P_n(x))^{\frac{n}{2}+1}.$$
$$B_n(x) = P_n(-x))^{\frac{n}{2}+1} + (P_n(x))^{\frac{n}{2}+1}.$$
$$S_n(x) = \frac{A_n}{B_n}.$$

Then, for $n \geq 1$,

(1) *If* $1 \leq x \leq 2^n$ *then* $0 \leq 4P_n(x) < -P_n(-x)$.
(2) *If* $1 \leq x \leq 2^n$ *then* $1 \leq S_n < \frac{5}{3}$.
(3) *If* $-2^n \leq x \leq -1$ *then* $-\frac{5}{3} < S_n(x) \leq -1$.

PROOF. (1) Since $x \geq 1$, we have that $P_n(x) \geq 0$. Clearly, $x-1 < x+1$, and $(x-2^i)^2 < (-x-2^i)^2$ for $i = 1, \ldots, n$. Also, if $2^{k-1} \leq x < 2^k$ then $4(x-2^k)^2 \leq 4.2^{2k-2} = 2^{2k} < (-x-2^k)^2$. Together these imply that $4P_n(x) < -P_n(-x)$.

(2) If $P_n(x) = 0$ then $S_n(x) = 1$. If $P_n(x) \neq 0$, then we can write

$$S_n(x) = 1 + \frac{2}{\left(\left(\frac{-P_n(-x)}{P_n(x)}\right) - 1\right)}.$$

Simple algebra and part (1) yield the desired result.

(3) This follows from (2) and the fact that $S_n(x)$ is an odd function, i.e., $S_n(-x) = -S_n(x)$.

□

THEOREM 2.44. PL *is closed under union.*

PROOF. Fix two languages D and E in PL. We will show that $D \cup E$ is also in PL.

Let f_D and f_E be the functions given by Proposition 2.41 for D and E, respectively. By Lemma 2.1, let $n = n(|x|)$ be a polynomial such that 2^n bounds the maximum absolute value of $f_D(x)$ and $f_E(x)$. Define $A_D(x) = A_n(f_D(x))$. Similarly, define $B_D(x), S_D(x), A_E(x), B_E(x)$, and $S_E(x)$. Note that $A_D(x), B_D(x), A_E(x)$, and $B_E(x)$ are GapL functions.

Let $H(x) = S_D(x) + S_E(x) + 1$. By Lemma 2.43, we have

(1) If $x \in D$ and $x \in E$ then $H(x) \geq 3$.
(2) If $x \in D$ and $x \notin E$ then $H(x) \geq \frac{1}{3}$.
(3) If $x \notin D$ and $x \in E$ then $H(x) \geq \frac{1}{3}$.
(4) If $x \notin D$ and $x \notin E$ then $H(x) \leq -1$.

Thus we have that $x \in D \cup E$ if and only if $H(x) > 0$.

We would be finished if H were a GapL function, but unfortunately it may even take nonintegral values. We do have

$$H(x) = \frac{A_D(x)}{B_D(x)} + \frac{A_E(x)}{B_E(x)} + 1 = \frac{A_D(x)B_E(x) + A_E(x)B_D(x) + B_D(x)B_E(x)}{B_D(x)B_E(x)}.$$

Note that for nonzero integers p and q we have $p/q > 0$ if and only if $pq > 0$. Then we define

$$H'(x) = (A_D(x)B_E(x) + A_E(x)B_D(x) + B_D(x)B_E(x))(B_D(x)B_E(x)).$$

We have that

(1) $H'(x)$ is GapL function.
(2) For all $x \in \Sigma^*$, $H(x) > 0$ if and only if $H'(x) > 0$.
(3) $x \in D \cup E$ if and only if $H'(x) > 0$.

Finally applying Proposition 2.41 to H', we have $D \cup E \in$ PL. □

COROLLARY 2.45. PL *is closed under intersection.*

EXERCISE 1. *Define* $\mathrm{PL}^{\mathrm{PL}}$.

DEFINITION 2.46. **(Low-Degree Polynomials to Approximate the Sign Function)** Let m and r be positive integers. Define:

$$P_m(z) = (z-1)\prod_{1\le i\le m}(z-2^i)^2. \tag{2.1}$$

$$Q_m(z) = -P_m(z) - P_m(-z). \tag{2.2}$$

$$A_{m,r}(z) = (Q_m(z))^{2r}. \tag{2.3}$$

$$B_{m,r}(z) = (Q_m(z)^{2r}) + (2P_m(z))^{2r}. \tag{2.4}$$

$$R_m, r(z) = \left(\frac{2P_m(z)}{Q_m(z)}\right)^{2r}. \tag{2.5}$$

$$S_{m,r}(z) = (1+R_{m,r}(z))^{-1}. \tag{2.6}$$

$R_{m,r}(z)$ and $S_{m,r}(z)$ are two auxiliary functions that will help us understand the properties of $A_{m,r}(z)$ and $B_{m,r}(z)$.

LEMMA 2.47. (1) *For all positive integers m and r, $S_{m,r}(z) = \frac{A_{m,r}(z)}{B_{m,r}(z)}$.*

(2) *For every positive integers m and r, both $A_{m,r}(z)$ and $B_{m,r}(z)$ are polynomials in z of degree $O(rm)$.*

(3) *For all integers $m, r \ge 1$ and every integer z,*

(a) *if $1 \le z \le 2^m$, then $1 - 2^{-r} \le S_{m,r}(z) \le 1$, and*

(b) *if $-2^m \le z \le -1$, then $0 \le S_{m,r}(z) \le 2^{-r}$.*

PROOF. The proofs of (1) and (2) are by routine calculation. We leave them to the reader. To prove (3), let m and r be positive integers. First consider the case when $1 \le z \le 2^m$. In this case $P_m(z) \ge 0$ and $P_m(-z) < 0$. We prove the following claim.

CLAIM 2.48. *If $1 \le z \le 2^m$, then $0 \le P_m(z) < -\frac{P_m(-z)}{9}$.*

Proof of Claim. The claim clearly holds for $z = 1$. So suppose that $1 \le z \le 2^m$. There is a unique i, $1 \le i \le m$, such that $2^i \le z < 2^{i+1}$. Let t be that i. Then (i) $2^t \le z$ and (ii) $z/2 < 2^t$. By combining (i) and (ii) we get $0 \le (z - 2^t) < \frac{z}{2}$, and from (ii) we get $z + 2 > \frac{3z}{2}$, and thus, $\frac{z}{2} < |-z-2^t|/3$. By combining the two inequalities, we have $(z-2^t)^2 < \frac{(-z-2^t)^2}{9}$. Note that $z - 1 < z + 1$ and $|z - 2^i| \le z + 2^i$ for every i, $1 \le i \le m$. Thus, in light of the definition of P_m, $P_m(z) < -\frac{P_m(-z)}{9}$.

Now by the above claim $0 \le P_m(z) < -\frac{P_m(-z)}{9}$. Combining this with $P_m(-z) \le 0$ yields $Q_m(z) > -\frac{8P_m(-z)}{9} > 0$. Thus

$$0 \le R_{m,r}(z) < \left(\frac{2\left(\frac{-1}{9}\right)P_m(-z)}{-\frac{8}{9}P_m(-z)}\right)^{2r} = \left(\frac{1}{4}\right)^{2r} < 2^{-r}.$$

Since $R_{m,r}(z) \ge 0$ and since for ever $\delta \ge 0$, $(1+\delta) > 0$ and $(1+\delta)(1\delta) = 1 - \delta^2 \le 1$, we have

$$1 \ge S_{m,r}(z) = \frac{1}{1+R_{m,r}(z)} > 1 - R_{m,r}(z) > 1 - 2^{-r}.$$

Hence (3a) holds.

Next consider the case when $-2^m \le z \le -1$. For this range of values of z, in light of Claim 2.48 we have $0 \le P_m(-z) < -\frac{P_m(z)}{9}$. This implies $0 < Q_{m,r}(z) < -P_m(z)$. Thus

$$R_{m,r}(z) \ge \left(\frac{2P_m(z)}{-P_m(z)}\right)^{2r} = 4^r > 2^r.$$

Hence (3b) holds. □

LEMMA 2.49. *For each $L \in$ PL and each polynomial r, there exists* GapL *functions $g : \Sigma^* \to \mathbb{N}$ and $h : \Sigma^* \to \mathbb{N}^+$ such that, for all $x \in \Sigma^*$,*

(1) *if $\chi_L(x) = b$, then $1 - 2^{-r(|x|)} \leq \frac{g(\langle x,b\rangle)}{h(x)} \leq 1$, and*
(2) *if $\chi_L(x) \neq b$, then $0 \leq \frac{g(\langle x,b\rangle)}{h(x)} \leq 2^{-r(|x|)}$.*

PROOF. Let $L \in$ PL and f be a GapL function witnessing, in the sense of Proposition 2.41, the membership of L in PL. Then, for every $x \in \Sigma^*$, the absolute value of $f(x)$ is at least one. Let m be a polynomial such that, for every $x \in \Sigma^*$, the absolute value of $f(x)$ is at most $2^{m(|x|)}$. Let r be an arbitrary polynomial. Define:

$$\begin{aligned} h(x) &= B_{m(|x|),r(|x|)}(f(x)), \\ g(\langle x,1\rangle) &= A_{m(|x|),r(|x|)}(f(x)), \textit{and} \\ g(\langle x,0\rangle) &= B_{m(|x|,r(|x|)}(f(x)) - A_{m(|x|),r(|x|)}(f(x)). \end{aligned}$$

Then, for every $x \in \Sigma^*$, $g(\langle x,0\rangle) + g(\langle x,1\rangle) = h(x)$. For every $x \in \Sigma^*$, by Part 1 of Lemma 2.47, $S_{m(|x|),r(|x|)}(f(x)) = \frac{g(\langle x,1\rangle)}{h(x)}$ and since $g(\langle x,0\rangle) + g(\langle x,1\rangle) = h(x)$, $1 - S_{m(|x|,r(|x|)}(f(x)) = \frac{g(x,0)}{h(x)}$. So, by Lemma 2.47 and the first claim of the provious sentence, $1 - 2^{-r(|x|)} \leq \frac{g(\langle x,1\rangle)}{h(x)} \leq 1$ if $f(x) > 0$ and $2^{-r(|x|)} \geq \frac{g(\langle x,1\rangle)}{h(x)} \geq 0$ if $f(x) < 0$. Since $\frac{g(\langle x,0\rangle)}{h(x)} = 1 - \frac{g(\langle x,1\rangle)}{h(x)}$, we have $1 - 2^{-r(|x|)} \leq \frac{g(\langle x,1\rangle)}{h(x)} \leq 1$ if $f(x) < 0$ and $2^{-r(|x|)} \geq \frac{g(\langle x,1\rangle)}{h(x)} \geq 0$ if $f(x) > 0$. Now it remains to prove that both g and h are in GapL, but this is easy to prove because of the closure properties of GapL under addition and multiplication of upto polynomial length. We leave that verification as an exercise for the reader. □

THEOREM 2.50. $\mathrm{PL}^{\mathrm{PL}} = \mathrm{PL}$.

PROOF. Let $L \in \mathrm{PL}^{\mathrm{PL}}$ such that N is a non-deterministic oracle Turing machine that takes space at most $O(\log n)$ on inputs of length n and which makes oracle queries to a language $A \in$ PL to correctly decide if an input string $x \in \Sigma^*$ is in L or not, where Σ is the input alphabet. Let p be a polynomial bounding the running time of N. Let q be a polynomial such that, for every $x \in \Sigma^*$,

- $acc_{N^A}(x) \leq 2^{q(|x|)}$, and
- $x \in L \Leftrightarrow acc_{N^A}(x) \geq 2^{q(|x|)-1}$.

Let m be a polynomially bounded function, as defined above, that maps each integer n to the number of queries that N makes on each input of length n. Then $m(n) \leq p(n)$ for all n. For each $x \in \Sigma^*$ and w, $|w| = m(|x|)$, let $\alpha(x,w)$ denote the number of accepting computation paths that M on input x would have if for every i, $1 \leq i \leq m(|x|)$, the oracle answer to the i^{th} query of N on input x is taken to be affirmative if the i^{th} bit of w is a 1 and otherwise is taken to be negative. Then, for every $x \in \Sigma^*$, $\alpha(x, \Gamma_N^A(x)) = acc_{N^A}(x)$, and for every $w \in \Sigma^{m(|x|)}$, $\alpha(x, \Gamma_N^A(x)) \leq 2^{q(|x|)}$. We now define a GapL function d such that on any given input string $x \in \Sigma^*$, we have $x \in L$ if and only if $d(x) \geq 0$.

We assume without loss of generality that N is a $O(\log n)$ space bounded oblivious oracle Turing machine which obeys the Ruzzo-Simon-Tompa oracle access mechanism as decsribed in Section 1.1.4.2 in Chapter 1 such that it generates queries in succession in the lexicographical order. We use $\Gamma_N^A(x)$ to denote the answer sequence that the oracle A provides to N on input x.

By Lemma 2.49, there exists non-negative function $g \in \text{GapL}$ and a strictly positive function $h \in \text{GapL}$ such that, for all $x \in \Sigma^*$ and $b \in \{0,1\}$,

$$\begin{aligned} 1 - 2^{-r(|x|)} \leq \frac{g(\langle x, b\rangle)}{h(x)} &\leq 1 \text{ if } \chi_A(x) = b, \text{ and} \\ 0 \leq \frac{g(\langle x, b\rangle)}{h(x)} &\leq 2^{-r(|x|)}, \text{ otherwise.} \end{aligned}$$

Define, for each $x \in \Sigma^*$,

$$s(x) = \Sigma_{|w|=m(|x|)} \alpha(x,w) \prod_{1 \leq i \leq m(|x|)} g(\langle y_{x,i}, w_i\rangle)$$

and

$$t(x) = \prod_{1 \leq i \leq m(|x|)} h(y_{x,i}).$$

We claim that for every $x \in \Sigma^*$,

$$x \in L \Leftrightarrow \frac{s(x)}{t(x)} \geq 2^{q(|x|)-1} - \frac{1}{4}.$$

To prove the claim let $x \in \Sigma^*$. First suppose that $x \in L$. For $w = \Gamma_N^A$, the fraction

$$\kappa(x,w) = \frac{\prod_{1 \leq i \leq m(|x|)} g(\langle y_{x,i}, w_i\rangle)}{t(x)}$$

is at least

$$1 - m(|x|)2^{-min\{r(|y_{x,1}|),\ldots,r(|y_{x,m(|x|)}|)\}}.$$

Because m is bounded by p, because r is a natural polynomial, and because the machine N is a length-increasing query generator, the above amount is at least

$$1 - p(|x|)2^{-p(|x|)-q(|x|)-1} > 1 - 2^{-q(|x|)-1}.$$

Thus the fraction $\frac{s(x)}{t(x)}$ is at least

$$2^{q(|x|)-1}(1 - 2^{-q(|x|)-1}) = 2^{q(|x|)-1} - \frac{1}{4}.$$

Next suppose that $x \notin L$. For $w = \Gamma_N^A$, $\kappa(x,w) \leq 1$ and $\alpha(x,w) = acc_{N^A}(x) \leq 2^{q(|x|)} - 1$. For other w of length $m(|x|)$, $\alpha(x,w) \leq 2^{q(|x|)}$ and

$$\kappa(x,w) \leq 2^{-min\{r(y_{x,1}),\ldots,r(|y_{x,m(|x|)}|)\}}.$$

Because m is bounded by p, because r is a natural polynomial, and because the machine N is a length-increasing query generator, the above amount is at most $2^{-p(|x|)-q(|x|)-1}$. Since the number of w, $|w| = m(|x|)$, such that $w \neq \Gamma_N^A(x)$ is $2^{m(|x|)} - 1 < 2^{p(|x|)}$, $\frac{s(x)}{t(x)}$ is less than

$$2^{q(|x|)-1} - 1 + 2^{p(|x|)}2^{-p(|x|)-q(|x|)-1} = 2^{q(|x|)-1} - 1 + 2^{-q(|x|)-1} \leq 2^{q(|x|)-1} - \frac{1}{2}.$$

Thus the claim holds.

Now define

$$d(x) = 4s(x) - (2^{q(|x|+1} - 1)t(x).$$

Then, for every $x \in \Sigma^*$, $x \in L$ if and only if $d(x) \geq 0$. Clearly $d \in \text{GapL}$ which proves our theorem. □

2.9. An alternate proof of NL $\subseteq$ PL

THEOREM 2.51. $C_{=}L \subseteq PL$.

PROOF. Let $L \subseteq \Sigma^*$ and let $L \in C_{=}L$. Let $f \in GapL$ be such that for any input $x \in \Sigma^*$ we have $x \in L$ if and only if $f(x) = 0$. Let $L_1 = \{x \in \Sigma^* | f(x) > 0\}$ and let $L_2 = \{x \in \Sigma^* | f(x) < 0\}$. It is easy to note that $x \in L$ if and only if both of the following two conditions are false:

1. is $f(x) > 0$, equivalently $x \in L_1$,
2. is $f(x) < 0$, equivalently $x \in L_2$.

Clearly $L_1 \in PL$. Determining if $f(x) > 0$ is therefore possible using a logarithmic space bounded deterministic Turing machine that has access to L_1 as an oracle. Next to determine if $f(x) < 0$ let us consider the function $h(x) = -(2f(x) + 1)$. Using Theorem 2.50 it follows that $h \in GapL$. We now observe that if $x \in L_2$ then $f(x) < 0$ and so $h(x) > 0$. Conversely if $x \notin L_2$ then $f(x) \geq 0$ and so $h(x) < 0$. As a result we get $L_2 \in$ co-PL. Using Corollary 2.42 it follows that $L_2 \in PL$. Consider $L' = L_1 \cup L_2$. Then it follows from Theorem 2.44 that $L' \in PL$. Now a logarithmic space bounded deterministic Turing machine that has access to L' as an oracle can therefore determine if the input $x \in \Sigma^*$ is such that $f(x) = 0$. This shows that $C_{=}L \subseteq L^{PL}$. However $L^{PL} = PL$ by Theorem 2.50 from which we get $C_{=}L \subseteq PL$. $\square$

Using Corollary 1.47 and Theorem 2.51 we therefore obtain another proof of Corollary 1.39 also.